FOXMCloud / TAMIZAJE

Documentación integral del proyecto

Funcionamiento, ramas, arquitectura y guía de archivos

Spring Boot · Vue.js · React · Bootstrap

Edición 15 de octubre de 2026
Código Vue de referencia: 9ddfda5

Índice navegable

1. Alcance y lectura del proyecto

- Qué resuelve el sistema
- Arquitectura de la variante Vue

2. Ramas y evolución del proyecto

- Historial verificable
- Comparación práctica
- Ejecutar la rama React
- Ejecutar el ZIP React
- Dependencias verificadas de la variante React
- Elegir y cambiar de rama sin mezclar versiones

3. Instalación, ejecución y distribución

- Obtener la versión Vue correcta
- Requisitos y modo local con H2
- Ejecutar con MySQL
- Empaquetar Vue en un solo JAR
- Qué representa build/ en este repositorio

4. Funcionamiento de la interfaz Vue

- Atención guiada en siete pasos
- Estado del asistente y conservación del borrador
- Guardado único, errores y reintentos
- Navegación, carga y listados
- Fichas existentes y controles
- Impresión, accesibilidad y alcance de demo
- Pruebas incluidas y cómo ejecutarlas

5. Backend, datos, reglas y API

- Recorrido de una atención completa
- Contrato de alta completa
- Modelo persistente y relaciones
- Valores enumerados
- Validaciones del alta completa
- Cálculos implementados
- API HTTP explícita
- Diferencias del contrato heredado
- Configuración, compilación y persistencia
- Pruebas Java y alcance

6. Mantenimiento y resolución de problemas

- Dónde realizar cada cambio
- Fallos habituales

Alcance actual y evolución
Regenerar el PDF

7. Catálogo de archivos y responsabilidades

Raíz y herramientas de compilación
Punto de entrada Java
Controladores HTTP
DTO y contratos de intercambio
Entidades del dominio
Valores enumerados
Repositorios JPA
Servicios de aplicación y cálculo
Perfiles y configuración de recursos
Frontend Vue y bibliotecas locales
Pruebas Java y JavaScript
Frontend React
Distribución ZIP de la variante React
Artefactos generados heredados de build/
Documentación nueva

8. Anexos y referencias

Contenido del ZIP de prueba
Glosario
Fuentes y límites de la revisión

1. Alcance y lectura del proyecto

Repositorio: github.com/foxmcloud/Tamizaje. **Fecha de revisión:** 15 de octubre de 2026, zona America/Argentina/Buenos_Aires. **Código de referencia de la variante actual:** `9ddfda591a779e94cbc7f9aecebefc9f35954e1f`.

Este manual explica el proyecto existente, sus cuatro ramas remotas, el recorrido de una atención, la arquitectura, la API, las reglas de cálculo y la función de cada archivo versionado. Se verificó el contenido con Git y con las fuentes; las diferencias entre ramas se describen sin atribuir a `main` funcionalidades que sólo existen en Vue o React. El commit que publique este PDF será posterior al código de referencia y añadirá documentación, sin cambiar ese funcionamiento.

La revisión encontró **139 rutas únicas** al reunir las cuatro ramas: 100 en `main`, 112 en React, 115 en distribución y 124 en Vue, con muchos archivos compartidos. De las 139 rutas, **53 son salidas heredadas de `build/`**. El catálogo también incluye los cinco archivos nuevos de este manual, por lo que el inventario adjunto describe **144 rutas**. Una ruta compartida se documenta una sola vez con sus ramas; sus variantes se explican en los capítulos correspondientes.

No se incluyen los objetos internos de `.git/`, los archivos temporales de herramientas, `node_modules/`, las cachés de Gradle o npm, los datos H2 creados al usar la aplicación ni el contenido interno completo de bibliotecas de terceros. Esos elementos no forman parte de las 139 rutas versionadas. Las cuatro entradas del ZIP de demostración y sus recursos web principales se explican en un anexo, diferenciándolas de archivos Git independientes.

Para quien usa la aplicación, conviene comenzar por el capítulo de instalación y el flujo Vue. Para quien la mantiene, seguir la arquitectura, el contrato HTTP y el catálogo. El índice inicial y los marcadores del PDF llevan a los capítulos; el texto y las rutas pueden buscarse con el lector PDF.

Qué resuelve el sistema

Tamizaje registra una evaluación preventiva: vincula un paciente y un enfermero, recoge medidas, presión, hábitos y antecedentes y almacena un resultado orientativo. El paciente se identifica con un ID, fecha de nacimiento y sexo; los nombres y apellidos corresponden al personal de salud. El sistema permite consultar fichas, registrar o corregir tomas de presión, recalcular e imprimir el resultado.

La variante que responde al pedido más reciente es **Vue + Bootstrap dentro del mismo proyecto Spring Boot**, con siete pasos y un guardado conjunto al finalizar. Las ramas React y demo permanecen como alternativas históricas. Las reglas de cálculo que se explican son las que implementa el código; no se presentan como una implementación completa del cuestionario FINDRISC estándar.

Arquitectura de la variante Vue



Confirmación final: POST /api/tamizajes/completo

Validar → vincular personas y controles → calcular → persistir en una transacción

El frontend y la API usan el mismo origen; no hay servidor npm en esta variante.

2. Ramas y evolución del proyecto

La documentación corresponde a los commits inspeccionados el 15 de octubre de 2026. La consulta del remoto confirmó cuatro ramas: `main`, React, demo y Vue, con los mismos commits documentados aquí. Los nombres `origin/*` son referencias locales al remoto. `work` es una rama exclusivamente local que apunta al mismo commit base que `origin/main`: no representa una implementación diferente ni una quinta rama publicada.

Rama o referencia	Commit inspeccionado	Propósito y estado funcional
`origin/main`	`f85d9f1`	Backend original Spring Boot + JPA + MySQL. No incluye el frontend React ni Vue.
`work`	`f85d9f1`	Referencia local al mismo contenido de `main`.
`codex/react-frontend-tamizaje`	`6bc8f38`	Frontend React con Bootstrap y React-Bootstrap en `frontend/`; durante desarrollo se ejecuta con Vite y usa la API Spring Boot.
`codex/tamizaje-bootstrap-demo`	`470c5bd`	Extiende la rama React únicamente con `distribucion/`, un ZIP listo para pruebas locales con un JAR que integra React compilado, API y H2.
`codex/vue-bootstrap-springboot`	`9ddfda5`	Variante actual: Vue y Bootstrap incluidos en el proyecto Spring Boot, asistente de atención de siete pasos, confirmación final, persistencia transaccional e informe.

El árbol real de commits se divide a partir del backend base. La variante Vue se creó directamente desde ese backend, sin integrar los commits React. La rama demo deriva de React.

```
f85d9f1  Backend original [main / work]
├── cd927af  Frontend React funcional
│   └── 6bc8f38  React + Bootstrap [codex/react-frontend-tamizaje]
│       └── 470c5bd  Paquete ZIP [codex/tamizaje-bootstrap-demo]
└── a0f14fb  Vue + Bootstrap dentro de Spring Boot
    └── 3b1b4d7  Atención completa en una transacción
        └── 9ddfda5  Asistente validado por pasos [codex/vue-bootstrap-springboot]
```

Los cambios del frontend no forman parte de `main` en la revisión documentada. Descargar el ZIP de `main` o clonar y permanecer en la rama predeterminada obtiene el backend original. Para probar la versión que guía enfermero, paciente y controles paso a paso debe elegirse la rama Vue.

Historial verificable

Commit	Cambio concreto
`f85d9f1086df12984abeea694c2accb898894164`	Carga inicial de backend y artefactos de compilación en `build/`.

Commit	Cambio concreto
`cd927af2e3872adc5a642f8f5f727311f940a7d4`	Agrega una aplicación React funcional y la consulta del cuestionario por ficha.
`6bc8f3809bc2c7bb0ec7be093a7b05e14fb3f757`	Adapta el frontend React a Bootstrap 5 y React-Bootstrap, incluidos modales accesibles.
`470c5bdb1e2c45fcb0eb92ec8379582734f176b3`	Agrega instrucciones, checksum y ZIP de prueba descargable.
`a0f14fb878295f2e6c0dff213026ada198085995`	Añade Vue 3 y Bootstrap locales a los recursos estáticos de Spring Boot, perfil H2 demo, documentación y pruebas web.
`3b1b4d7e1451b7ab123e5d3b6accb91698fb7eab`	Agrega el guardado conjunto de enfermero, paciente, controles y resultado, con validación y reversión transaccional.
`9ddfda591a779e94cbc7f9aecebefc9f35954e1f`	Transforma la carga de atención en un asistente con un paso visible, revisión, navegación y validación por etapa.

Comparación práctica

Aspecto	Backend original	React	ZIP de demostración React	Vue actual
Interfaz de gestión	Sin interfaz web propia	React en `frontend/`	React compilado en JAR	Vue en `src/main/resources/static/`
Framework visual	Ninguno	Bootstrap + React-Bootstrap	Bootstrap + React-Bootstrap	Bootstrap
Proceso de desarrollo	Spring Boot	Spring Boot + Vite	Un proceso Java	Un proceso Spring Boot
Puerto habitual	8080	API 8080; Vite 5173	8090	8080
Node.js	No	Sí	No para ejecutar el ZIP	No para ejecutar ni empaquetar; opcional para tests JS
Base predeterminada	MySQL 3307	MySQL 3307 del backend	H2 en `datos/`	MySQL 3307; perfil `demo` con H2 en `.tamizaje-demo/`
Alta de una atención	Peticiones separadas	Modales y ficha por controles	Mismo flujo que React	Asistente de siete pasos, guardado final atómico
Persistencia después del reinicio	Sí, en MySQL	Sí, en el backend	Sí, en archivos H2	Sí, en MySQL o archivos H2 del perfil demo
Incluye el asistente Vue	No	No	No	Sí

El nombre «demo» no implica datos temporales. Tanto el ZIP React como el perfil `demo` de Vue utilizan H2 en archivo y conservan los registros cuando se cierra y vuelve a iniciar la aplicación. La carpeta de datos se crea al ejecutar; no contiene pacientes precargados dentro del repositorio ni del ZIP.

Ejecutar la rama React

Requiere JDK 17, MySQL/MariaDB configurado como espera el backend y Node.js 22.12 o superior recomendado por la documentación del frontend. La configuración Java declara compatibilidad fuente 11, pero el código existente y el entorno se validaron con JDK 17; el wrapper Gradle es 7.6.4.

En una terminal, desde la raíz del repositorio:

```
bash gradlew bootRun
```

En otra terminal:

```
cd frontend
npm ci
npm run dev
```

El navegador abre el servidor Vite en el puerto 5173 de la máquina en ejecución. Vite reenvía `/api` al backend en el puerto 8080. La variable `API_TARGET` permite cambiar ese destino de desarrollo. `npm test` ejecuta las diez pruebas React y del cliente; `npm run build` produce `frontend/dist/`. En producción, esa carpeta requiere un servidor estático y un proxy del mismo origen para `/api`; el comando `vite preview` no configura ese proxy. Esta rama no integra automáticamente el build React en `bootJar`.

El flujo React permite dar de alta primero pacientes y personal, crear una ficha con consentimiento y cargar controles desde sus pestañas. Los guardados se realizan por separado. La modificación compartida del backend agrega GET del cuestionario para recuperar la información al reabrir una ficha. El asistente de pasos y POST de atención completa pertenecen a Vue.

Ejecutar el ZIP React

En la rama `codex/tamizaje-bootstrap-demo`, el archivo `distribucion/Tamizaje-bootstrap.zip` puede descargarse con el vínculo publicado en `distribucion/README.md`. Debe descomprimirse completo. Contiene exactamente cuatro archivos bajo `Tamizaje/`: `iniciar.sh`, `iniciar.cmd`, `LEEME.txt` y `tamizaje-demo.jar`. No incluye una instalación Java ni fuentes duplicados del frontend.

Con Java 17 disponible, Windows utiliza `iniciar.cmd` y macOS/Linux `bash iniciar.sh`. Los scripts cambian a su propia carpeta y ejecutan el JAR en `127.0.0.1:8090`, con H2 de archivo `./datos/tamizaje`, usuario `sa` y dialecto H2. Esos argumentos reemplazan los valores MySQL de `application.properties` incluido en el JAR. El archivo JAR contiene las dependencias y el resultado compilado de Vite; no requiere instalar Node.js o MySQL para ejecutarlo. La terminal debe permanecer abierta y Ctrl+C detiene el servidor.

El ZIP es una instantánea de la variante React de octubre de 2026. No se actualiza automáticamente al cambiar la rama Vue y no incluye su asistente, su endpoint transaccional ni su perfil `demo` de recursos fuente. Para entregar cambios nuevos debe reconstruirse expresamente el artefacto correspondiente.

El ZIP ocupa 39.140.363 bytes y contiene un JAR de 43.451.520 bytes. Su SHA-256 registrado es `4d3796093878f61d8973045896e35d844cdada96fe2918367ac1c8f430592f9f`. El JAR inspeccionado incluye Spring Boot 2.7.14, Spring MVC 5.3.29, Spring Data JPA 2.7.14, Hibernate 5.6.15.Final, MySQL Connector/J 8.0.33 y H2 2.1.214.

Dependencias verificadas de la variante React

`package.json` define rangos admitidos; `package-lock.json` define las versiones exactas que instala `npm ci`. Por eso declarar `^19.2.0` no significa que el paquete bloqueado sea 19.2.0.

Paquete	Rango en package.json	Versión bloqueada	Función
`react`	`^19.2.0`	19.3.0	Estado, componentes y renderizado de interfaz.
`react-dom`	`^19.2.0`	19.3.0	Montaje de React en el DOM del navegador.
`bootstrap`	`^5.3.8`	5.3.8	Grilla adaptable y estilos de componentes.
`react-bootstrap`	`^2.10.10`	2.10.10	Componentes Bootstrap implementados para React; administra modales, teclado y foco.
`lucide-react`	`^0.468.0`	0.468.0	Iconos de interfaz.
`@vitejs/plugin-react`	`^5.1.0`	5.2.0	Integración React en Vite.
`vite`	`^7.3.0`	7.3.7	Servidor de desarrollo y compilación de frontend.
`vitest`	`^3.2.0`	3.2.7	Ejecución de pruebas JavaScript/React.
`jsdom`	`^26.1.0`	26.1.0	DOM simulado para pruebas.
`@testing-library/react`	`^16.3.0`	16.3.3	Renderizado y consultas de componentes en pruebas.
`@testing-library/user-event`	`^14.6.0`	14.6.8	Interacciones de usuario simuladas.

Vue utiliza bibliotecas locales con versiones fijas distintas: Vue 3.5.44 y Bootstrap 5.3.8. No incorpora React, React-Bootstrap, Lucide React ni Vite. Los archivos `vendor/versions.json` y las licencias permiten identificar los recursos distribuidos.

Elegir y cambiar de rama sin mezclar versiones

Para continuar el proyecto solicitado actualmente, corresponde `codex/vue-bootstrap-springboot`. React se conserva como alternativa, y la rama de distribución conserva el paquete histórico. Cambiar de rama no cambia automáticamente el puerto, los procesos que ya están ejecutándose ni las carpetas de bases de datos.

Antes de cambiar, detener los servidores con Ctrl+C, revisar `git status` y guardar los cambios propios mediante un commit o un stash identificado. Luego, en una copia de trabajo limpia:

```
git fetch origin
git switch codex/vue-bootstrap-springboot
```

Si la rama aún no existe localmente:

```
git switch --track origin/codex/vue-bootstrap-springboot
```

Para comparar implementaciones sin cambiar la copia activa pueden utilizarse `git show rama:ruta` y `git diff origin/main..rama`. Para probar varias variantes al mismo tiempo conviene crear copias separadas con `git worktree add`, asignar puertos distintos y mantener independientes los archivos H2. No debe compartirse una misma base H2 de archivo entre dos procesos Java simultáneos.

Al regresar a React se ejecuta `npm ci` dentro de `frontend/`; al regresar a Vue se ejecuta Spring Boot desde la raíz. Para distribuir una variante se reconstruye su JAR o build a partir de la rama correcta, ya que los binarios de `build/` que venían en el backend original no representan necesariamente el contenido actual de las fuentes.

3. Instalación, ejecución y distribución

Obtener la versión Vue correcta

Para una copia nueva:

```
git clone --branch codex/vue-bootstrap-springboot https://github.com/foxmcloud/Tamizaje.git
cd Tamizaje
```

La rama predeterminada `main` contiene sólo el backend original. Para una copia existente, consultar primero `git status`, guardar los cambios propios y elegir la rama correspondiente. Si una configuración de Git sólo descarga `main`, puede ampliarse la descarga con `git fetch origin '+refs/heads/*:refs/remotes/origin/*'`; luego se puede usar `git switch --track origin/codex/vue-bootstrap-springboot` cuando aún no exista esa rama local.

Requisitos y modo local con H2

Usar **JDK 17** para trabajar con el wrapper Gradle 7.6.4. `java -version` permite comprobar qué Java está usando la terminal. La compatibilidad de fuentes 11 declarada en Gradle y el JDK empleado para ejecutar Gradle son conceptos distintos. El primer arranque necesita descargar Gradle y dependencias de Maven; el navegador usa las bibliotecas Vue y Bootstrap ya incluidas en el repositorio.

macOS / Linux, desde la raíz:

```
bash gradlew bootRun --args='--spring.profiles.active=demo'
```

Windows, CMD o PowerShell:

```
gradlew.bat bootRun --args="--spring.profiles.active=demo"
```

Después de `Started TamizajeApplication`, abrir `http://localhost:8080` en el navegador de la computadora que ejecuta Java. Mantener la terminal abierta. Ctrl+C detiene el proceso. El perfil demo escucha en loopback y conserva los registros en `.tamizaje-demo/`, relativa al directorio desde el que se ejecuta Java. No exige Node.js ni MySQL. Esta dirección es una instrucción de ejecución local, no un enlace de acceso a una instancia de nube.

Ejecutar con MySQL

Sin el perfil demo se utiliza MySQL en el puerto 3307 y la base `tamizaje`, según `application.properties`. La base debe existir y el usuario configurado debe tener acceso. La configuración de desarrollo del repositorio usa `root` con contraseña vacía. Para otra instalación, reemplazar los datos mediante variables de entorno de Spring:

Variable	Qué configura
`SPRING_DATASOURCE_URL`	URL JDBC; por ejemplo `jdbc:mysql://localhost:3306/tamizaje?useSSL=false&serverTimezone=UTC` si el servidor usa 3306.

Variable	Qué configura
`SPRING_DATASOURCE_USERNAME`	Usuario de la base configurada.
`SPRING_DATASOURCE_PASSWORD`	Contraseña de ese usuario; no hace falta incorporarla al repositorio.

Iniciar con `bash gradlew bootRun`, o su equivalente Windows. Hibernate usa `ddl-auto=update`: ajusta tablas según entidades, pero no reemplaza una estrategia de migraciones y copias de seguridad. La variante actual no incorpora scripts de migración versionados.

Empaquetar Vue en un solo JAR

```
bash gradlew bootJar
java -jar build/libs/tamizaje-0.0.1-SNAPSHOT.jar --spring.profiles.active=demo
```

En Windows, cambiar `bash gradlew` por `gradlew.bat`. El JAR contiene frontend, API y dependencias. Para MySQL se omite el argumento de perfil demo y se configura la conexión. Conservar o respaldar la carpeta de datos H2 por separado: no está incluida dentro del JAR ni se reemplaza automáticamente al actualizarlo.

En desarrollo `bootRun` usa `sourceResources` y permite recargar los recursos estáticos editados. Cuando se ejecuta un JAR, cambiar HTML/JS/CSS en `src/` no modifica el binario ya iniciado: se debe detener esa ejecución, reconstruir y volver a iniciar. No sobrescribir el JAR mientras el proceso lo utiliza.

Qué representa build/ en este repositorio

El commit inicial incluyó 53 archivos de compilación e informes dentro de `build/`. Son productos generados, no otra implementación. Pueden contener clases anteriores a Vue y resultados históricos de pruebas. Las fuentes de referencia están en `src/` y la configuración en Gradle. `.gitignore` evita nuevas salidas, pero no elimina lo que ya estaba rastreado.

El entorno de nube usado durante el desarrollo guardó herramientas en `/workspace/tooling` y compilaciones/datos de trabajo en `/workspace/runtime`; esas rutas son externas al repositorio y no son requisitos del proyecto en una computadora local. El manual no incorpora sus logs, bases o cachés al catálogo del proyecto.

4. Funcionamiento de la interfaz Vue

La rama `codex/vue-bootstrap-springboot` contiene el frontend dentro de `src/main/resources/static`. Spring Boot sirve tanto la página como los recursos y los endpoints `/api` desde un mismo proceso y un mismo puerto. No hay un segundo servidor de frontend: al iniciar Spring Boot, ya se puede abrir la aplicación en el navegador. El JAR generado con `bootJar` incluye la interfaz, las bibliotecas y el backend.

Vue 3.5.44 se importa como módulo ES desde un archivo local. La distribución `vue.esm-browser.prod.js` incluye el compilador necesario para interpretar la plantilla HTML en el navegador. Bootstrap 5.3.8 aporta estilos, grilla, componentes y comportamiento de los modales; `app.css` define la identidad visual, el asistente por pasos, la adaptación a pantallas pequeñas y la impresión. No se necesita Node.js, npm ni Vite para ejecutar o empaquetar esta rama. Node.js 18 o superior se utiliza únicamente si se quieren ejecutar las pruebas JavaScript incluidas.

Las bibliotecas no se descargan desde una CDN al abrir la página. `vendor/versions.json` registra versiones, origen y hashes SHA-256 de los archivos incluidos. Las licencias MIT de Vue y Bootstrap se conservan junto a cada biblioteca.

Atención guiada en siete pasos

La pantalla inicial se llama **Registrar atención** y muestra una sola sección por vez. Los botones **Continuar** y **Anterior**, el indicador “Paso N de 7” y la barra de progreso orientan el recorrido.

Paso	Qué se ingresa	Qué controla la interfaz
1. Enfermero	Un enfermero nuevo con nombre, apellido y turno, o uno existente.	Nombre y apellido no vacíos, de hasta 80 caracteres; turno `MAÑANA`, `TARDE` o `VESPERTINO`. Si se reutiliza un registro, exige un ID válido. El selector sólo muestra profesionales con rol `ENFERMERO`.
2. Paciente	Un paciente nuevo con fecha de nacimiento y sexo, o uno existente.	Fecha real del calendario; edad de 14 a 120 años; sexo `F`, `M` u `O`. Si se reutiliza un paciente, exige un ID válido.
3. Atención	Código de ficha, consentimiento y supervisor opcional.	Código no vacío, consentimiento aceptado explícitamente y supervisor válido cuando se selecciona. El selector sólo muestra profesionales con rol `SUPERVISOR`.
4. Medidas	Peso, talla y cintura.	Peso entre 30 y 300 kg; talla entre 120 y 220 cm; cintura obligatoria en este asistente. El campo HTML admite de 1 a 300 cm. Los campos permiten décimas.
5. Presión	Presión sistólica y diastólica de la primera toma.	Valores enteros; sistólica entre 70 y 260 mmHg; diastólica entre 40 y 150 mmHg; sistólica mayor que diastólica. Una lectura elevada válida puede registrarse: el cálculo del servidor determina su clasificación.
6. Hábitos y antecedentes	Actividad física, frutas y verduras, medicación, tabaquismo y antecedentes familiares.	Cada respuesta debe completarse; las tres primeras aceptan Sí o No explícitos. Si hay antecedentes, exige al menos un tipo: diabetes, hipertensión, enfermedad cardiovascular u otro.
7. Revisar y guardar	Resumen de todos los datos y confirmación final.	Vuelve a validar cada sección. Los botones **Editar** regresan a la sección correspondiente antes de guardar.

El código inicial se propone con el formato `TAM-AAAAMMDD-SUFIJO`, con fecha obtenida de `toISOString()` y un sufijo aleatorio; puede modificarse. La plantilla limita el campo a 100 caracteres, mientras que el validador compartido y el contrato de alta admiten hasta 255. Si se automatiza la API, no debe confundirse el límite del campo visual con el límite del contrato.

El modelo de paciente no contiene nombre, apellido, documento, dirección ni teléfono. En esta versión se identifica al paciente por su ID, fecha de nacimiento y sexo; los selectores e informes muestran su número, edad y sexo. Nombre y apellido pertenecen al profesional de salud. La documentación no presupone campos que el proyecto todavía no implementa.

Estado del asistente y conservación del borrador

`freshDirectForm()` construye el borrador inicial. `directForm` contiene todos los datos ingresados, mientras que `directStep` indica el paso visible y `directMaxStep` el paso más lejano alcanzado. Los botones de pasos futuros permanecen deshabilitados hasta recorrerlos. Es posible volver a un paso anterior sin perder lo ingresado; al saltar nuevamente hacia adelante, se revalidan las secciones intermedias. Cambiar entre crear o reutilizar una persona conserva los borradores de ambas opciones, pero sólo se envía la opción activa.

Los campos de cada sección se renderizan mediante `v-if`. Su desaparición visual al avanzar no borra el estado reactivo. **Continuar** ejecuta `validateDirectRegistrationStep()` sobre el paso actual y luego la validación HTML del formulario visible. El guardado final revisa nuevamente los seis pasos de ingreso en orden; si detecta un error, vuelve al primero que requiere corrección y coloca el foco en el mensaje.

El borrador vive en la memoria del navegador. Navegar a otro listado y regresar conserva el estado mientras siga abierta esa instancia de la aplicación. Recargar o cerrar la página descarta el borrador no guardado: no hay autoguardado, almacenamiento local ni ficha parcial creada al avanzar. **Nueva atención** reinicia el asistente; después de guardar, reutiliza el enfermero recién utilizado cuando sigue presente en el listado cargado.

Guardado único, errores y reintentos

Al confirmar **Guardar atención y calcular resultado**, el frontend construye un único JSON y envía `POST /api/tamizajes/completo`. El cuerpo expresa cada persona como ID existente o datos nuevos, nunca ambas alternativas a la vez. Incluye código, consentimiento, supervisor opcional, antropometría, presión y cuestionario. La normalización transforma números y booleanos de los controles HTML en los tipos esperados por Java, recorta nombres y código y elimina selecciones de antecedentes incompatibles con “Sin antecedentes”.

El backend realiza el alta completa en una transacción y devuelve la ficha vinculada, las personas, los controles y el resultado. Si falla la validación o una operación de esa transacción, el alta no deja personas ni controles parciales. Mientras se guarda, el estado `busy` bloquea los botones para evitar envíos repetidos desde la interfaz. Si ocurre un error, el borrador permanece para corregirlo o reintentar. Los mensajes del servidor se conservan para explicar, por ejemplo, un código ya utilizado.

La transacción protege la integridad del servidor, pero no reemplaza una confirmación de entrega. Si se pierde la conexión después de que el servidor confirmó la transacción, el cliente puede no recibir el resultado; el código único evita crear otra atención con el mismo código y un reintento puede informar duplicidad. En ese caso se debe consultar el listado para verificar si la ficha quedó guardada.

Después de recibir un alta correcta, desaparece el asistente y se muestra el informe con enfermero responsable, paciente, código, medidas y presión. Incluye FINDRISC, riesgo cardiovascular, semáforos de IMC y presión, semáforo global, observaciones y condición de derivación. Una derivación inmediata se presenta con un aviso destacado. **Abrir ficha** lleva a la consulta detallada; **Imprimir informe** abre el diálogo de impresión; **Nueva atención** inicia la siguiente atención.

Navegación, carga y listados

La aplicación ofrece **Registro por pasos**, **Resumen**, **Tamizajes**, **Pacientes** y **Equipo de salud**. El resumen muestra cantidades de fichas, pacientes y profesionales, además de las cinco fichas con ID más alto. Los listados tienen búsqueda local sobre los datos ya descargados; no implementan paginado de servidor.

`reload()` pide en paralelo `/api/pacientes`, `/api/enfermeros` y `/api/tamizajes`. Un contador de versión evita que una respuesta antigua reemplace una recarga posterior. Los estados de carga, error y éxito se presentan en pantalla y la carga fallida ofrece **Reintentar**. El asistente bloquea sus acciones cuando la carga inicial no está disponible.

La navegación usa fragmentos de URL como `#registro`, `#pacientes` y `#tamizajes/123`; `setRoute()` reemplaza el fragmento con `history.replaceState`. Al iniciar o recargar se interpreta el fragmento: una ficha puede reabrirse por ID. No se utiliza Vue Router ni se mantiene un historial completo de transiciones; los botones internos controlan la navegación.

Los listados de pacientes y equipo permiten altas independientes mediante modales Bootstrap. El equipo admite roles enfermero y supervisor. La acción principal **Nuevo tamizaje** abre el asistente. En el código permanece una rama de modal antiguo para fichas, pero la función `openModal('tamizaje')` la redirige al flujo por pasos; no constituye el recorrido principal del usuario.

Fichas existentes y controles

Al abrir una ficha, `loadRecord()` obtiene en paralelo antropometría, tomas de presión, cuestionario y resultado. En los controles opcionales, una respuesta HTTP 404 significa que aún no se registró ese dato. Otros errores se muestran y bloquean cambios hasta actualizar la ficha. Se comprueba el ID de ficha al recibir respuestas para evitar aplicar datos de otra ficha abierta mientras la petición estaba en curso.

La vista conserva cuatro pestañas: **Antropometría**, **Presión**, **Cuestionario** y **Resultado**. Es una pantalla de consulta y seguimiento, distinta del alta guiada. La antropometría y el cuestionario permiten creación si no existen; una vez guardados se muestran en modo de lectura porque la API de esos recursos no implementa edición. El formulario histórico permite omitir cintura; la API representa una cintura no tomada como cero. El asistente nuevo exige ingresarla.

En presión arterial se pueden agregar, editar y eliminar tomas. El alta utiliza `POST /api/tamizajes/{id}/tomas-presion`; la edición, `PUT /api/tomas-presion/{idControl}`; y la eliminación, `DELETE` sobre esa última ruta, previa confirmación del navegador. La interfaz permite cancelar una edición y limpia el formulario cuando la operación termina correctamente. Antes de mutar controles comprueba consentimiento y que la ficha esté cargada sin errores.

Para calcular el resultado de una ficha existente se exige consentimiento, antropometría, cuestionario y al menos una toma de presión. `POST /api/tamizajes/{id}/resultado` calcula o recalcula. La interfaz muestra clasificaciones recibidas del backend; no reproduce el algoritmo clínico en JavaScript.

El resultado es un cálculo persistido. Si se cambia un control desde la ficha actual, `resultOutdated` marca el informe como pendiente de recalcular y deshabilita la impresión hasta actualizarlo. Esta marca existe en memoria del frontend: no es un control global de versiones que detecte cambios de otros navegadores ni se conserva al recargar. Al recalcular se limpia la marca. Si la ficha modificada coincide con el informe del alta recién guardada, se reinicia ese informe para que no siga disponible una copia anterior en la pantalla de registro.

Impresión, accesibilidad y alcance de demo

La impresión usa `window.print()` y reglas `@media print`. Se ocultan menú, botones, navegación, modales y asistente; el informe conserva sus datos, categorías y advertencias, incluida la derivación inmediata. El usuario puede elegir una impresora o guardar como PDF mediante el diálogo de su navegador. La vista se adapta a móviles: los pasos se reducen a indicadores numerados, las columnas se reorganizan y las tablas usan contenedores desplazables.

La plantilla incluye etiquetas asociadas a campos, encabezados, enlace para saltar al contenido, roles de alerta y progreso, indicación de paso actual, foco en el encabezado o error y retorno del foco al cerrar un modal. Son ayudas implementadas en la interfaz; no sustituyen una auditoría formal de accesibilidad.

El perfil `demo` usa H2 en archivo bajo `.tamizaje-demo/`, escucha en `127.0.0.1` y deshabilita la consola H2. No es una base temporal en memoria: sus registros persisten entre arranques. Los tests web y de alta completa reemplazan esa conexión por bases H2 en memoria. Para demo se deben usar datos ficticios. La aplicación todavía no incluye autenticación; los roles de profesionales almacenados no constituyen por sí solos permisos de acceso.

El informe declara la variante FINDRISC efectivamente usada: cintura y antecedentes de glucosa elevada no se incorporan al puntaje. Guardar cintura no significa que participe en ese cálculo. El profesional de salud debe interpretar la evaluación mostrada.

Pruebas incluidas y cómo ejecutarlas

Las pruebas JavaScript usan el runner integrado de Node.js y no requieren instalar dependencias. `api.test.mjs` contiene 13 casos del cliente HTTP, errores, formatos y validadores; `direct-form.test.mjs`, 10 del contrato de alta completa; y `wizard-form.test.mjs`, 11 de validación por pasos y revisión final. Son 34 casos declarados. Estos tests importan el módulo de validación; no simulan por sí solos todos los clics y diseños de un navegador.

En Java, `WebUiIntegrationTests` contiene 9 ejecuciones contando parámetros: página inicial, recursos locales, API del mismo origen y manejo de un cuestionario inexistente. `TamizajeCompletoIntegrationTests` contiene 28 ejecuciones contando parámetros: relaciones, reutilización de personas, resultado equivalente al recálculo, errores de validación, consentimiento, roles, duplicidad y reversión de transacciones. `TamizajeApplicationTests` contiene la prueba original de carga del contexto y usa la configuración predeterminada, por lo que puede requerir MySQL. Describir estas pruebas no implica haberlas vuelto a ejecutar para generar esta documentación.

```
# Validaciones JavaScript, Node.js 18 o superior
node --test src/test/js/*.test.mjs

# Integración web y alta completa con H2, sin MySQL
bash gradlew test --tests cent35.WebUiIntegrationTests --tests cent35.TamizajeCompletoIntegrationTests

# Suite Java completa; incluye contexto con la configuración por defecto
bash gradlew test
```

Para ejecutar la aplicación local se recomienda JDK 17 con el wrapper Gradle 7.6.4, aunque `build.gradle` declara compatibilidad de código fuente Java 11:

```
bash gradlew bootRun --args='--spring.profiles.active=demo'
```

En Windows se usa `gradlew.bat` en lugar de `bash gradlew`. Cuando aparezca `Started TamizajeApplication`, se abre `http://localhost:8080` en la misma computadora. La terminal debe seguir abierta. En desarrollo, `bootRun` sirve los recursos fuente mediante `sourceResources`; para distribuir cambios en un JAR se debe ejecutar de nuevo `bootJar`.

5. Backend, datos, reglas y API

El backend se encuentra en `src/main/java/cent35`. Es una aplicación Spring Boot 2.7.14 con Spring MVC, Spring Data JPA, Hibernate y Spring Data REST. La aplicación principal es `TamizajeApplication`: descubre componentes del paquete `cent35`, configura el servidor web integrado y arranca el contexto de Spring. Su `CommandLineRunner` está vacío; no crea pacientes ni datos iniciales.

La organización separa cinco responsabilidades: `controllers` publica las rutas HTTP explícitas; `dtos` define entradas y respuestas JSON; `services` coordina el alta completa y los cálculos; `models` mantiene las entidades y reglas del dominio; `repositories` permite consultar y persistir mediante JPA. La interfaz Vue y sus bibliotecas locales se sirven desde `src/main/resources/static`, dentro del mismo proceso y JAR. No se necesita otro servidor de frontend para esta variante.

Recorrido de una atención completa

1. Vue reúne los datos y envía una sola petición JSON a `POST /api/tamizajes/completo` al confirmar la revisión final.
2. `TamizajeCompletoController` deserializa el cuerpo en `TamizajeCompletoRequest` y llama a `TamizajeCompletoService.registrar`.
3. El servicio exige consentimiento verdadero, normaliza el identificador y comprueba que no esté utilizado. Resuelve enfermero, paciente y supervisor opcional; crea los controles en memoria y aplica sus validaciones.
4. Construye un `Tamizaje` y vincula ambos lados de sus relaciones. La primera toma de presión recibe el número 1. `ResultadoTamizajeService`, apoyado en `FindriscService` y los métodos de `Tamizaje`, calcula el resultado antes de la primera escritura.
5. El método `registrar` está anotado con `@Transactional`. Guarda solamente las personas nuevas y luego ejecuta `saveAndFlush` sobre el tamizaje; las asociaciones con cascada guardan antropometría, cuestionario, presión y resultado. Una excepción revierte las escrituras de esta atención.
6. Devuelve HTTP 201 con `TamizajeCompletoDTO`, que contiene tamizaje, paciente, operador, antropometría, toma de presión, cuestionario y resultado. La respuesta usa DTO para evitar serializar directamente relaciones JPA circulares.

El identificador también tiene una restricción `unique` en la base de datos. Si dos solicitudes compiten por el mismo código, el controlador reconoce la violación de integridad después de que el servicio transaccional se haya revertido y responde 409 cuando el identificador ya existe. Esta garantía de alta coordinada corresponde al endpoint `/completo`; los endpoints heredados guardan cada entidad por separado.

Contrato de alta completa

`TamizajeCompletoRequest` admite `idRegistro`, `consent`, `operadorId` o `operador`, `pacienteId` o `paciente`, `supervisorId`, `antropometria`, `tomaPresion` y `cuestionario`. Para cada persona se debe enviar exclusivamente el identificador existente o el objeto nuevo. Enviar ambos o ninguno devuelve 400.

```
{
  "idRegistro": "TAM-EJEMPLO-001",
  "consent": true,
  "operador": {
    "nombre": "Ana",
    "apellido": "Pérez",
    "turno": "MAÑANA"
  },
  "paciente": {
    "fechaNacimiento": "1976-01-01",
    "sexo": "F"
  },
  "supervisorId": null,
  "antropometria": { "pesoKg": 70, "tallacm": 170, "cinturacm": 80 },
  "tomaPresion": { "sistolicaMmHg": 120, "diastolicaMmHg": 80 },
  "cuestionario": {
    "actividadFisica": true,
    "frutasVerduras": true,
    "medicacion": false,
    "tabaquismo": "NUNCA",
    "antecedenteFamiliar": "NO",
    "tiposAntecedentes": []
  }
}
```

Para reutilizar una persona se reemplaza su objeto por `"operadorId": 1` o `"pacienteId": 1`, utilizando un identificador existente real. No se envían IDs de controles ni resultados: el servidor crea esas entidades. La respuesta completa no incluye un bloque de supervisor, aunque la relación opcional sí se guarda.

Modelo persistente y relaciones

Entidad	Datos y relaciones
`Paciente`	ID entero, fecha de nacimiento, sexo y fecha de registro. Un paciente puede tener muchos tamizajes. Calcula la edad con la fecha actual del servidor y valida 14 a 120 años inclusive. No almacena nombre, documento ni dirección del paciente.
`Enfermero`	ID entero, nombre, apellido, rol y turno. Mantiene dos colecciones de tamizajes: operados y supervisados. Los roles son datos del dominio, no un mecanismo de inicio de sesión.
`Tamizaje`	ID largo, código de registro único, consentimiento y fecha/hora. Pertenece a un paciente, un operador y un supervisor opcional. Tiene un cuestionario, una antropometría, varias tomas de presión y un resultado.
`Control`	Entidad abstracta con ID largo `idControl`. Usa herencia JPA `JOINED`: cada control tiene una fila en la tabla base y otra en la tabla de su subtipo.
`Antropometria`	Subtipo de control con peso, talla y cintura; calcula IMC. Su FK al tamizaje es obligatoria y única.
`TomaPresion`	Subtipo de control con número de toma, sistólica y diastólica. Varias tomas pueden pertenecer al mismo tamizaje. El resultado utiliza la toma válida con mayor `numeroToma`.

Entidad	Datos y relaciones
`Cuestionario`	Subtipo de control con actividad física, consumo de frutas/verduras, medicación, tabaquismo y antecedentes. Los tipos de antecedente se guardan mediante `@ElementCollection`. Tiene una relación uno a uno obligatoria con el tamizaje.
`ResultadosTamizaje`	Puntaje y categoría FINDRISC, riesgo cardiovascular simplificado, semáforos, derivación y observaciones. Su FK al tamizaje es obligatoria y única. Tiene un campo interno de semáforo de cintura, pero el cálculo actual no lo establece ni el DTO lo publica.

`Tamizaje` aplica `CascadeType.ALL` a cuestionario, antropometría, tomas y resultado. La colección de presión usa además `orphanRemoval=true`. Los métodos `vincularCuestionario`, `vincularAntropometria`, `addTomaPresion` y `vincularResultados` completan las referencias inversas para mantener consistente el grafo. La relación con paciente y enfermeros no aplica cascada de persistencia; el servicio guarda expresamente las personas nuevas.

Valores enumerados

Enum	Valores JSON admitidos
`Sexo`	`M`, `F`, `O`
`RolEnfermero`	`ENFERMERO`, `SUPERVISOR`
`Turnos`	`MAÑANA`, `TARDE`, `VESPERTINO`
`Tabaquismo`	`NUNCA`, `EX`, `ACTUAL`
`AntecedentesFamiliares`	`NO`, `SEGUNDO_GRADO`, `PRIMER_GRADO`
`TipoAntecedente`	`DIABETES`, `HIPERTENSION`, `ENFERMEDAD_CARDIOVASCULAR`, `OTRO`
`SemaforoGlobal`	`VERDE`, `AMARILLO`, `ROJO`

Los enums se persisten como texto mediante `EnumType.STRING`. Las etiquetas visibles pueden traducirse en la interfaz, pero las peticiones deben conservar estos valores exactos.

Validaciones del alta completa

Dato	Regla implementada en backend
Consentimiento	Debe ser exactamente verdadero. Falta, falso o nulo produce 403.
Identificador	Texto no vacío después de quitar espacios externos, hasta 255 caracteres y único. Duplicado produce 409.
Enfermero nuevo	Nombre y apellido no vacíos, hasta 255 caracteres cada uno, turno obligatorio. El servicio asigna el rol `ENFERMERO`.
Enfermero existente	Debe existir y tener rol `ENFERMERO`. Supervisor existente opcional debe tener rol `SUPERVISOR`.

Dato	Regla implementada en backend
Paciente	Fecha de nacimiento y sexo presentes; edad actual entre 14 y 120 años inclusive, también al reutilizar un paciente.
Peso y talla	Peso entre 30 y 300 kg inclusive; talla entre 120 y 220 cm inclusive.
Cintura	En el endpoint completo admite omisión, nulo o cero como medición no tomada. Si se informa, debe ser finita y estar entre 0 y 300 cm inclusive. No participa del cálculo FINDRISC actual.
Presión	Sistólica entera entre 70 y 260; diastólica entera entre 40 y 150; sistólica estrictamente mayor que diastólica.
Cuestionario	Los tres booleanos, tabaquismo y antecedente familiar son obligatorios. Si el antecedente familiar no es `NO`, debe haber al menos un tipo de antecedente. La lista no puede ser nula ni contener elementos nulos.

La interfaz puede aplicar restricciones adicionales para guiar la captura. Las reglas anteriores describen el contrato del servicio completo; no deben atribuirse de forma automática a todos los controladores heredados.

Cálculos implementados

Esta sección describe literalmente el comportamiento del código para mantenimiento y revisión. El proyecto calcula una variante de FINDRISC basada en los datos disponibles y un riesgo cardiovascular simplificado; no calcula una probabilidad cardiovascular porcentual.

IMC. `Antropometria.calcularIMC` calcula `pesoKg / (tallacm / 100)²`. `Tamizaje.clasificarIMC` asigna amarillo a IMC menor que 18,5 o entre 25 inclusive y 30 exclusivo; rojo a IMC mayor o igual que 30; verde al resto del rango válido.

FINDRISC disponible. `FindriscService` exige paciente con fecha de nacimiento, antropometría válida y cuestionario válido. No utiliza perímetro de cintura ni antecedente de glucemia elevada, aunque el formulario actual permite registrar cintura. El sexo tampoco suma puntos en esta implementación.

Factor	Puntos
Edad menor de 45 años	0
Edad de 45 a 54 años	2
Edad de 55 a 64 años	3
Edad desde 65 años	4
IMC menor de 25	0
IMC entre 25 y 30 inclusive	1
IMC estrictamente mayor que 30	3
No realiza actividad física (`actividadFisica=false`)	2
No consume frutas/verduras (`frutasVerduras=false`)	1

Factor	Puntos
Informa medicación (`medicacion=true`)	2
Antecedentes familiares de segundo grado	3
Antecedentes familiares de primer grado	5

El límite exacto de IMC 30 recibe un punto FINDRISC por la condición `> 30` usada en el servicio, mientras que el semáforo de IMC ya es rojo en `>= 30`. Es una particularidad existente que conviene conservar visible al revisar reglas. El máximo de los factores implementados es 17 puntos. La clasificación contempla también `MUY_ALTO` por encima de 20, aunque esta categoría no se alcanza sumando únicamente los factores actuales.

Puntaje FINDRISC	Categoría
Menor de 7	`BAJO`
7 a 11	`LIGERAMENTE_ELEVADO`
12 a 14	`MODERADO`
15 a 20	`ALTO`
Mayor de 20	`MUY_ALTO`
Datos insuficientes	`DATOS_INSUFICIENTES`

Presión. Se selecciona la toma válida de mayor número. Es roja si sistólica >= 140 o diastólica >= 90; amarilla si no fue roja y sistólica >= 130 o diastólica >= 85; verde en el resto. Sistólica >= 180 o diastólica >= 110 activa `derivacionInmediata`.

Riesgo cardiovascular simplificado. Cuenta un factor por tabaquismo actual, uno por presión roja, uno por IMC >= 30 y uno por tener una lista de tipos de antecedentes no vacía. Devuelve el texto `MODO_SIMPLIFICADO_FACTORES_N`. Asigna rojo cuando hay `ENFERMEDAD_CARDIOVASCULAR` entre los tipos, o al menos tres factores; amarillo con uno o dos; verde con cero. Este conteo no usa un modelo de probabilidad a diez años.

Semáforo global. Es rojo si cualquier semáforo de IMC, presión o riesgo cardiovascular es rojo, o FINDRISC >= 15. Es amarillo si no fue rojo y alguno es amarillo, o FINDRISC >= 12. En los demás casos es verde. La derivación general se activa por semáforo rojo o por derivación inmediata. El servicio asigna observaciones sobre aviso al docente responsable para urgencia, consulta a centro de salud para derivación general, o mantenimiento de hábitos saludables.

La edad se calcula con `LocalDate.now()`, no con la fecha almacenada en la atención. Por eso un recálculo posterior a un cumpleaños puede producir un puntaje de edad distinto al del alta. El endpoint de recálculo reutiliza el resultado existente en lugar de crear otro resultado para el mismo tamizaje.

API HTTP explícita

Todas las rutas siguientes proceden de anotaciones de los controladores. GET devuelve JSON salvo errores de texto. Los cuerpos de error escritos por estos controladores son mensajes de texto; errores de deserialización o parámetros inválidos pueden usar la respuesta estándar de Spring.

Método y ruta	Entrada y finalidad	Respuestas relevantes
`GET /api/enfermeros`	Lista de `EnfermeroDTO`.	200
`GET /api/enfermeros/{id}`	Un enfermero. El método retorna `null` si no existe; no construye un 404 explícito.	200; cuerpo vacío al no existir según el comportamiento de retorno del controlador
`POST /api/enfermeros`	Parámetros `nombre`, `apellido`, `rol`, `turno`; alta individual. No recibe JSON.	201; 403 por campos vacíos según su validación; 400 por parámetros/enums inválidos
`GET /api/pacientes`	Lista de `PacienteDTO`.	200
`POST /api/pacientes`	Parámetros `fecha_nacimiento` (yyyy-MM-dd) y `sexo`; alta individual.	201; 403 por datos nulos según su validación; 400 por parámetros/formato inválidos
`GET /api/tamizajes`	Lista de fichas resumidas `TamizajeDTO`.	200
`GET /api/tamizajes/{id}`	Ficha resumida por ID.	200, 404
`POST /api/tamizajes`	Entidad JSON para inicializar una ficha individual; exige consentimiento y código.	201, 400, 403, 409
`POST /api/tamizajes/completo`	`TamizajeCompletoRequest`; alta coordinada con personas, controles y resultado.	201; 400 datos inválidos; 403 consentimiento; 404 persona inexistente; 409 código duplicado
`GET /api/tamizajes/{idTamizaje}/antropometria`	Control antropométrico del tamizaje.	200, 404
`POST /api/tamizajes/{idTamizaje}/antropometria`	JSON con peso/talla/cintura; guarda un control y devuelve DTO con IMC.	201, 400, 403, 404
`GET /api/cuestionarios`	Lista de todos los cuestionarios.	200
`GET /api/cuestionarios/{id}`	Cuestionario por ID de control.	200, 404
`GET /api/tamizajes/{idTamizaje}/cuestionario`	Cuestionario asociado a una ficha.	200, 404
`POST /api/tamizajes/{idTamizaje}/cuestionario`	JSON con respuestas y antecedentes.	201, 400, 404
`GET /api/tamizajes/{idTamizaje}/tomas-presion`	Lista ordenada por número de toma. No comprueba la existencia de la ficha; puede devolver una lista vacía.	200

Método y ruta	Entrada y finalidad	Respuestas relevantes
`GET /api/tomas-presion/{id}`	Presión por ID de control.	200, 404
`POST /api/tamizajes/{idTamizaje}/tomas-presion`	JSON con sistólica/diastólica; numera la toma con el conteo actual + 1.	201, 400, 404
`PUT /api/tomas-presion/{id}`	Reemplaza los valores sistólico/diastólico de una toma existente.	200, 400, 404
`DELETE /api/tomas-presion/{id}`	Elimina una toma existente.	204, 404
`GET /api/tamizajes/{idTamizaje}/resultado`	Obtiene el resultado guardado.	200, 404
`POST /api/tamizajes/{idTamizaje}/resultado`	Sin cuerpo; valida ficha y recalcula/guarda el resultado mediante el servicio compartido.	200, 400, 403, 404

Además, el proyecto incluye `spring-boot-starter-data-rest` y marca los repositorios con `@RepositoryRestResource`. Spring Data REST puede exportar sus recursos y relaciones automáticamente. La tabla anterior documenta la API explícita que usa la aplicación, no es una enumeración de los recursos HAL generados por el framework. Antes de integrar un cliente adicional conviene descubrir y comprobar esos recursos en la instancia concreta.

Diferencias del contrato heredado

La creación individual de paciente no llama a `Paciente.validar`; la edad se exige en el alta completa y en el recálculo. Las altas individuales de cuestionario y presión no hacen la misma comprobación explícita de consentimiento que los endpoints de antropometría y resultado. El alta individual de antropometría sólo valida peso y talla en el modelo; la validación adicional de cintura pertenece al servicio completo. Los POST individuales de antropometría/cuestionario no son operaciones de actualización: cada ficha tiene una relación única a esos controles y repetir la creación puede violar la restricción de la base.

Modificar o eliminar presión no recalcula automáticamente un resultado ya guardado. El cliente debe solicitar nuevamente `POST .../resultado` cuando corresponda. `TamizajeDTO` es intencionalmente resumido en el código actual: contiene `id`, `consent`, `timestamp` e `id_registro`; no incluye los IDs de paciente, operador o supervisor ni los controles. Las rutas explícitas no incluyen un CRUD completo para todas las entidades.

Configuración, compilación y persistencia

`build.gradle` aplica los plugins Java, Spring Boot 2.7.14 y dependency-management 1.1.4. Declara compatibilidad de código fuente Java 11, aunque el entorno de trabajo utiliza Java 17 para ejecutar Gradle 7.6.4. Incluye starters de web, JPA y Data REST, drivers MySQL/H2 en ejecución, y Spring Boot Test/JUnit Platform para pruebas. No incluye Spring Security, un sistema de autenticación ni migraciones Flyway/Liquibase.

`settings.gradle` define el nombre Gradle `tamizaje`. `gradle/wrapper/gradle-wrapper.properties` fija la distribución 7.6.4. `gradlew`, `gradlew.bat` y el JAR del wrapper arrancan esa distribución en Unix o Windows. El artefacto de `bootJar` lleva las clases y recursos web de la aplicación, además de sus dependencias.

`application.properties` fija el puerto 8080 y, sin perfil adicional, conecta a MySQL en `localhost:3307/tamizaje`, con usuario `root` y contraseña vacía de configuración de desarrollo. Usa el driver MySQL y dialecto MySQL8. Hibernate trabaja con `ddl-auto=update`: actualiza el esquema según entidades, no ejecuta migraciones versionadas. El nombre de aplicación configurado es `2026`.

`application-demo.properties` sobreescribe el datasource con H2 persistente en `./.tamizaje-demo/tamizaje`, modo de compatibilidad MySQL, usuario `sa` y dialecto H2. Fija escucha en `127.0.0.1`, conserva `ddl-auto=update` y desactiva la consola H2. El puerto sigue heredando 8080. Es un perfil de prueba local con datos ficticios; no necesita instalar MySQL. Las rutas relativas de datos se resuelven desde el directorio donde se inicia Java.

Pruebas Java y alcance

`WebUiIntegrationTests` arranca Spring con perfil demo y H2 en memoria. Mediante MockMvc comprueba página inicial, recursos locales Vue/Bootstrap/JS/CSS, lectura de pacientes por el mismo origen y 404 de cuestionario inexistente.

`TamizajeCompletoIntegrationTests` usa otra H2 en memoria y prueba el grafo persistido, vínculos de personas/controles, reutilización de personas, supervisor opcional, coincidencia entre alta y recálculo, datos inválidos, consentimiento, roles, personas inexistentes, fallos de cálculo y reversión por código duplicado, incluida una simulación de carrera contra la restricción única de base. También verifica cintura devuelta/persistida y sus variantes omitida/nula/cero.

`TamizajeApplicationTests` es la prueba original de carga de contexto. No activa perfil demo: si se ejecuta sin sobreescrituras, depende de la configuración MySQL predeterminada. Los dos grupos nuevos pueden ejecutarse independientemente con `--tests cent35.WebUiIntegrationTests --tests cent35.TamizajeCompletoIntegrationTests`.

Las pruebas verifican comportamiento técnico de la implementación. No constituyen una validación clínica independiente de las reglas ni cobertura exhaustiva de todos los endpoints de Spring Data REST.

6. Mantenimiento y resolución de problemas

Dónde realizar cada cambio

Necesidad	Archivos principales y efecto
Cambiar textos o campos Vue	`static/index.html`, estado y revisión en `static/js/app.js`, validadores en `static/js/api.js`. Si cambia el contrato, actualizar también los DTO/servicios Java.
Cambiar estilo o impresión	`static/css/app.css`. Preservar la información del resultado que debe imprimirse y la adaptación móvil.
Agregar una regla de alta completa	`TamizajeCompletoService`, `TamizajeCompletoRequest` y pruebas de integración; la interfaz debe guiar la misma regla sin sustituir la validación del servidor.
Cambiar puntaje o clasificación	`FindriscService` y reglas del modelo `Tamizaje`; el servicio `ResultadoTamizajeService` centraliza alta y recálculo. Revisar pruebas y documentar los límites nuevos.
Agregar un dato persistente	Entidad de `models`, DTO, contrato/controlador/servicio, formulario y pruebas. Considerar las relaciones JPA y la evolución de datos existentes.
Cambiar React	Rama React: `frontend/src/App.jsx`, `frontend/src/api.js` y `frontend/src/styles.css`; usar `npm ci`, tests y build propios de esa variante.
Actualizar bibliotecas Vue/Bootstrap	Reemplazar archivos oficiales de `static/vendor`, conservar licencias y actualizar versiones/hashes de `versions.json`. No editar bibliotecas minificadas para cambiar el producto.
Actualizar este manual	Editar `docs/documentacion-integral.md`, mantener el inventario JSON y regenerar el PDF con `docs/generar_pdf.py`.

Las plantillas Vue usan interpolaciones `{{ ... }}` dentro de HTML. Un formateador HTML genérico puede insertar un salto de línea dentro de un literal JavaScript y romper su compilación. Por ese motivo `index.html` está excluido de Prettier; conservar en una línea los literales de esas expresiones.

Fallos habituales

Síntoma	Comprobación y acción
Sólo aparece backend o no está la pantalla por pasos	Revisar `git branch --show-current`; el asistente pertenece a la rama Vue y no a `main` ni al ZIP React.
Java/Gradle no inicia	Comprobar `java -version` y `JAVA_HOME`; usar JDK 17 para el wrapper fijado. Si falla la primera descarga, comprobar acceso a Gradle/Maven.
Puerto 8080 o 8090 ocupado	Identificar la instancia que ya está en ejecución. Cerrar su terminal con Ctrl+C o configurar otro `server.port`; la demo ZIP usa 8090 y Vue 8080.

Síntoma	Comprobación y acción
Falla la conexión MySQL	Revisar que el servidor esté disponible, el puerto 3307 o el configurado, la existencia de la base y las variables del datasource. Para probar sin MySQL usar el perfil demo de Vue.
H2 informa archivo bloqueado	Comprobar que no haya dos procesos usando el mismo archivo de datos. Cerrar la instancia anterior o usar carpetas de datos separadas.
Datos no guardados al avanzar de paso	Es el diseño del asistente: se guardan sólo al confirmar la revisión. Recargar antes de guardar descarta el borrador en memoria.
Identificador duplicado, HTTP 409	Elegir un código distinto o consultar si ya existe la ficha. Si hubo una interrupción después del envío, puede haberse confirmado aunque el navegador no recibiera la respuesta.
Resultado anterior después de editar presión	Recalcular desde la ficha. Los cambios no disparan recálculo automático; el indicador local de informe pendiente no detecta cambios de otros navegadores.
Cambios estáticos no se ven al ejecutar un JAR	Reconstruir ese JAR y reiniciarlo; comprobar la rama y el archivo realmente iniciado.
Error al ejecutar la suite Java completa sin MySQL	La prueba de contexto original usa la configuración base. Ejecutar los dos grupos nuevos con H2, o preparar la base requerida para toda la suite.

Alcance actual y evolución

No se implementan autenticación, cuentas de usuario, permisos por sesión ni auditoría de cambios. `ENFERMERO` y `SUPERVISOR` representan roles del registro de atención. Tampoco existen nombres/DNI de pacientes, paginación del servidor en los listados del frontend, edición de antropometría/cuestionario por la API explícita ni sincronización de resultados entre navegadores. Estas ausencias se documentan para interpretar correctamente el sistema existente.

Los umbrales, la omisión de cintura/glucemia previa en el puntaje y el uso de edad actual al recalcular están descritos en el backend. Cualquier evolución clínica o de contratos requiere una revisión explícita: el manual no modifica esas reglas. El perfil local y el ZIP están pensados para pruebas con datos ficticios.

Regenerar el PDF

El generador requiere Python 3, el paquete Python Playwright, Pandoc y Chromium o Chrome disponibles en la computadora. No utiliza el servidor Spring Boot, Node.js ni la base de datos. Desde la raíz:

```
python3 -m pip install playwright
python3 docs/generar_pdf.py
# Si Chrome tiene otra ruta:
python3 docs/generar_pdf.py --chromium /ruta/al/ejecutable
```

La opción `--no-sandbox` se reserva para contenedores que ya proporcionan aislamiento. En una computadora habitual se omite. El script genera `docs/Tamizaje-documentacion-integral.pdf` a partir de la fuente Markdown y su CSS; para conservar marcadores, PDF etiquetado y pies de página se recomienda una versión actual de Chromium. Se comprueba que el archivo exista y tenga contenido; la revisión visual sigue siendo necesaria cuando cambia el formato.

7. Catálogo de archivos y responsabilidades

Cada ruta tiene una descripción individual. La columna de ramas indica **M** = main, **R** = React, **D** = ZIP/demo y **V** = Vue. La presencia de un mismo nombre en varias ramas no garantiza que su contenido sea idéntico. Las descripciones del backend se refieren a la variante Vue cuando hay diferencias.

Cambios compartidos relevantes: `CuestionarioController.java` agrega GET por ficha en R/D/V, ausente en M; `ResultadoTamizajeController.java` delega en un servicio nuevo sólo en V; `AntropometriaDTO.java` expone cintura en V; `build.gradle` agrega H2 como dependencia fuente en V. El JAR del ZIP contiene una configuración binaria de H2 propia, distinta de ese cambio de fuentes. Las fechas y SHAs auditados están en el capítulo de ramas y en el inventario JSON.

Raíz y herramientas de compilación

Archivo y ramas	Responsabilidad
`.gitignore` V	Excluye caché Gradle, nuevas salidas build/, datos locales H2 y logs en la rama Vue. No elimina los artefactos build/ que ya estaban versionados. La rama React utiliza su propia selección de exclusiones.
`.prettierignore` V	Conserva el formato original y hashes de vendor/. Excluye index.html del formateador genérico: una interpolación Vue con un literal de texto no debe romperse en varias líneas.
`HELP.md` M, R, D, V	Ayuda inicial generada por Spring Initializr con enlaces generales. Algunos vínculos apuntan a Spring Boot 4.1.1; la versión real del proyecto es la 2.7.14 definida en build.gradle.
`README.md` V	Guía de uso de la rama Vue integrada: requisitos, arranque con H2 demo o MySQL, JAR único, flujo de siete pasos, consulta y recálculo de fichas, limitaciones de cálculo/autenticación y comandos de pruebas.
`build.gradle` M, R, D, V	Define proyecto Java/Spring Boot 2.7.14, compatibilidad de fuentes 11, dependencias web/JPA/DataREST/MySQL/H2, recursos de bootRun y tareas de pruebas JUnit Platform/bootJar.
`gradle/wrapper/gradle-wrapper.jar` M, R, D, V	Binario oficial del wrapper que descarga y arranca la versión de Gradle indicada por sus propiedades; no contiene lógica de negocio.
`gradle/wrapper/gradle-wrapper.properties` M, R, D, V	Fija distribución Gradle 7.6.4 y parámetros de descarga/almacenamiento del wrapper para una compilación reproducible.
`gradlew` M, R, D, V	Script de arranque del wrapper Gradle para sistemas Unix; permite ejecutar bootRun, test y bootJar con la versión fijada.
`gradlew.bat` M, R, D, V	Script de arranque del wrapper Gradle para Windows; permite ejecutar las mismas tareas desde CMD/PowerShell.

Archivo y ramas	Responsabilidad
`settings.gradle` M, R, D, V	Asigna al proyecto Gradle el nombre tamizaje.

Punto de entrada Java

Archivo y ramas	Responsabilidad
`src/main/java/cent35/TamizajeApplication.java` M, R, D, V	Punto de entrada de Spring Boot; inicia el servidor y el escaneo del paquete cent35. Declara un CommandLineRunner vacío, sin carga de datos iniciales.

Controladores HTTP

Archivo y ramas	Responsabilidad
`build/classes/java/main/cent35/controllers/AntropometriaController.class` M, R, D, V	Bytecode Java heredado de AntropometriaController.class, generado al compilar su fuente del mismo nombre. No se edita ni se toma como referencia del código actual; puede estar desactualizado respecto de src/.
`build/classes/java/main/cent35/controllers/CuestionarioController.class` M, R, D, V	Bytecode Java heredado de CuestionarioController.class, generado al compilar su fuente del mismo nombre. No se edita ni se toma como referencia del código actual; puede estar desactualizado respecto de src/.
`build/classes/java/main/cent35/controllers/EnfermeroController.class` M, R, D, V	Bytecode Java heredado de EnfermeroController.class, generado al compilar su fuente del mismo nombre. No se edita ni se toma como referencia del código actual; puede estar desactualizado respecto de src/.
`build/classes/java/main/cent35/controllers/PacienteController.class` M, R, D, V	Bytecode Java heredado de PacienteController.class, generado al compilar su fuente del mismo nombre. No se edita ni se toma como referencia del código actual; puede estar desactualizado respecto de src/.
`build/classes/java/main/cent35/controllers/ResultadoTamizajeController.class` M, R, D, V	Bytecode Java heredado de ResultadoTamizajeController.class, generado al compilar su fuente del mismo nombre. No se edita ni se toma como referencia del código actual; puede estar desactualizado respecto de src/.
`build/classes/java/main/cent35/controllers/TamizajeController.class` M, R, D, V	Bytecode Java heredado de TamizajeController.class, generado al compilar su fuente del mismo nombre. No se edita ni se toma como referencia del código actual; puede estar desactualizado respecto de src/.
`build/classes/java/main/cent35/controllers/TomaPresionController.class` M, R, D, V	Bytecode Java heredado de TomaPresionController.class, generado al compilar su fuente del mismo nombre. No se edita ni se toma como referencia del código actual; puede estar desactualizado respecto de src/.
`src/main/java/cent35/controllers/AntropometriaController.java` M, R, D, V	Publica GET/POST /api/tamizajes/{id}/antropometria. Busca la ficha, exige consentimiento al crear, valida peso/talla, guarda con AntropometriaRepository y devuelve AntropometriaDTO.

Archivo y ramas	Responsabilidad
`src/main/java/cent35/controllers/CuestionarioController.java` M, R, D, V	Controlador REST de cuestionarios. En main/work proporciona las operaciones originales de alta y consulta por control; no ofrece GET por ficha. Las ramas React/demo y Vue agregan GET /api/tamizajes/{idTamizaje}/cuestionario para recuperar el cuestionario vinculado, devolver su DTO o 404 si falta la ficha/cuestionario. La interfaz lo usa al reabrir una atención para evitar altas duplicadas.
`src/main/java/cent35/controllers/EnfermeroController.java` M, R, D, V	Publica listado, consulta por ID y alta individual de enfermeros; el POST recibe parámetros nombre/apellido/rol/turno, persiste Enfermero y consulta mediante EnfermeroDTO.
`src/main/java/cent35/controllers/PacienteController.java` M, R, D, V	Publica listado y alta individual de pacientes; el POST recibe fecha_nacimiento y sexo como parámetros, agrega fecha de registro y guarda con PacienteRepository. Este alta individual no llama a Paciente.validar.
`src/main/java/cent35/controllers/ResultadoTamizajeController.java` M, R, D, V	Publica GET/POST /api/tamizajes/{id}/resultado. Valida consentimiento y controles, reutiliza el resultado existente, delega el cálculo a ResultadoTamizajeService y devuelve ResultadosTamizajeDTO.
`src/main/java/cent35/controllers/TamizajeCompletoController.java` V	Publica POST /api/tamizajes/completo para el asistente. Delega en el servicio transaccional, convierte errores previstos a HTTP y reconoce códigos duplicados tras una reversión por integridad.
`src/main/java/cent35/controllers/TamizajeController.java` M, R, D, V	Publica listado, consulta y creación individual de fichas. Exige consentimiento/código único al crear y devuelve TamizajeDTO resumido en las consultas.
`src/main/java/cent35/controllers/TomaPresionController.java` M, R, D, V	Publica listado de tomas por ficha y GET/POST/PUT/DELETE de presión. Valida sistólica/diastólica, asigna número por conteo + 1 al crear y devuelve TomaPresionDTO.

DTO y contratos de intercambio

Archivo y ramas	Responsabilidad
`build/classes/java/main/cent35/dtos/AntropometriaDTO.class` M, R, D, V	Bytecode Java heredado de AntropometriaDTO.class, generado al compilar su fuente del mismo nombre. No se edita ni se toma como referencia del código actual; puede estar desactualizado respecto de src/.
`build/classes/java/main/cent35/dtos/CuestionarioDTO.class` M, R, D, V	Bytecode Java heredado de CuestionarioDTO.class, generado al compilar su fuente del mismo nombre. No se edita ni se toma como referencia del código actual; puede estar desactualizado respecto de src/.
`build/classes/java/main/cent35/dtos/EnfermeroDTO.class` M, R, D, V	Bytecode Java heredado de EnfermeroDTO.class, generado al compilar su fuente del mismo nombre. No se edita ni se toma como referencia del código actual; puede estar desactualizado respecto de src/.

Archivo y ramas	Responsabilidad
`build/classes/java/main/cent35/dtos/PacienteDTO.class` M, R, D, V	Bytecode Java heredado de PacienteDTO.class, generado al compilar su fuente del mismo nombre. No se edita ni se toma como referencia del código actual; puede estar desactualizado respecto de src/.
`build/classes/java/main/cent35/dtos/ResultadosTamizajeDTO.class` M, R, D, V	Bytecode Java heredado de ResultadosTamizajeDTO.class, generado al compilar su fuente del mismo nombre. No se edita ni se toma como referencia del código actual; puede estar desactualizado respecto de src/.
`build/classes/java/main/cent35/dtos/TamizajeDTO.class` M, R, D, V	Bytecode Java heredado de TamizajeDTO.class, generado al compilar su fuente del mismo nombre. No se edita ni se toma como referencia del código actual; puede estar desactualizado respecto de src/.
`build/classes/java/main/cent35/dtos/TomaPresionDTO.class` M, R, D, V	Bytecode Java heredado de TomaPresionDTO.class, generado al compilar su fuente del mismo nombre. No se edita ni se toma como referencia del código actual; puede estar desactualizado respecto de src/.
`src/main/java/cent35/dtos/AntropometriaDTO.java` M, R, D, V	Proyección JSON de una antropometría: idControl, pesoKg, tallacm, cinturacm e imc calculado. Evita devolver el vínculo JPA inverso al tamizaje.
`src/main/java/cent35/dtos/CuestionarioDTO.java` M, R, D, V	Proyección JSON de cuestionario: ID, tres respuestas booleanas, tabaquismo, grado familiar y copia de tipos de antecedentes; excluye la referencia inversa a la ficha.
`src/main/java/cent35/dtos/EnfermeroDTO.java` M, R, D, V	Proyección JSON de enfermero con ID, nombre, apellido, rol y turno. No serializa sus colecciones de fichas operadas/supervisadas.
`src/main/java/cent35/dtos/PacienteDTO.java` M, R, D, V	Proyección JSON de paciente con id_paciente, fecha_nacimiento, sexo y fecha_registro. No serializa la colección de tamizajes.
`src/main/java/cent35/dtos/ResultadosTamizajeDTO.java` M, R, D, V	Respuesta JSON del resultado: ID, puntaje/categoría FINDRISC, riesgo cardiovascular simplificado, semáforos, derivaciones y observaciones; usa nombres camelCase y excluye semáforo de cintura.
`src/main/java/cent35/dtos/TamizajeCompletoDTO.java` V	Agrupa la respuesta del alta atómica: DTO de ficha, paciente, operador, antropometría, primera presión, cuestionario y resultado; no incluye un bloque de supervisor.
`src/main/java/cent35/dtos/TamizajeCompletoRequest.java` V	Contrato JSON de entrada del alta completa. Define clases anidadas para personas/controles, identificadores alternativos para reutilizar personas y supervisor opcional; el servicio valida la exclusividad.
`src/main/java/cent35/dtos/TamizajeDTO.java` M, R, D, V	Proyección resumida de ficha con id, consent, timestamp e id_registro. No expone IDs de personas ni los controles relacionados.

Archivo y ramas	Responsabilidad
src/main/java/cent35/dtos/TomaPresionDTO.java M, R, D, V	Proyección JSON de presión con idControl, numeroToma, sistolicaMmHg y diastolicaMmHg; excluye la referencia a Tamizaje.

Entidades del dominio

Archivo y ramas	Responsabilidad
build/classes/java/main/cent35/models/Antropometria.class M, R, D, V	Bytecode Java heredado de Antropometria.class, generado al compilar su fuente del mismo nombre. No se edita ni se toma como referencia del código actual; puede estar desactualizado respecto de src/.
build/classes/java/main/cent35/models/Control.class M, R, D, V	Bytecode Java heredado de Control.class, generado al compilar su fuente del mismo nombre. No se edita ni se toma como referencia del código actual; puede estar desactualizado respecto de src/.
build/classes/java/main/cent35/models/Cuestionario.class M, R, D, V	Bytecode Java heredado de Cuestionario.class, generado al compilar su fuente del mismo nombre. No se edita ni se toma como referencia del código actual; puede estar desactualizado respecto de src/.
build/classes/java/main/cent35/models/Enfermero.class M, R, D, V	Bytecode Java heredado de Enfermero.class, generado al compilar su fuente del mismo nombre. No se edita ni se toma como referencia del código actual; puede estar desactualizado respecto de src/.
build/classes/java/main/cent35/models/Paciente.class M, R, D, V	Bytecode Java heredado de Paciente.class, generado al compilar su fuente del mismo nombre. No se edita ni se toma como referencia del código actual; puede estar desactualizado respecto de src/.
build/classes/java/main/cent35/models/ResultadosTamizaje.class M, R, D, V	Bytecode Java heredado de ResultadosTamizaje.class, generado al compilar su fuente del mismo nombre. No se edita ni se toma como referencia del código actual; puede estar desactualizado respecto de src/.
build/classes/java/main/cent35/models/Tamizaje.class M, R, D, V	Bytecode Java heredado de Tamizaje.class, generado al compilar su fuente del mismo nombre. No se edita ni se toma como referencia del código actual; puede estar desactualizado respecto de src/.
build/classes/java/main/cent35/models/TomaPresion.class M, R, D, V	Bytecode Java heredado de TomaPresion.class, generado al compilar su fuente del mismo nombre. No se edita ni se toma como referencia del código actual; puede estar desactualizado respecto de src/.
src/main/java/cent35/models/Antropometria.java M, R, D, V	Entidad JPA que extiende Control; almacena peso, talla y cintura, valida peso 30–300/talla 120–220, calcula IMC y se vincula uno a uno a Tamizaje.
src/main/java/cent35/models/Control.java M, R, D, V	Entidad abstracta base de antropometría, cuestionario y presión; define idControl e implementa herencia JPA JOINED con un método validar por defecto.

Archivo y ramas	Responsabilidad
`src/main/java/cent35/models/Cuestionario.java` M, R, D, V	Entidad de respuestas que extiende Control. Persiste enums y lista de tipos con ElementCollection, exige respuestas y tipos ante antecedentes familiares y se vincula uno a uno a la ficha.
`src/main/java/cent35/models/Enfermero.java` M, R, D, V	Entidad de personal con nombre, apellido, rol, turno y colecciones inversas de tamizajes operados y supervisados. Los roles son datos, no autenticación.
`src/main/java/cent35/models/Paciente.java` M, R, D, V	Entidad de paciente con nacimiento, sexo y fecha de alta; calcula edad respecto de LocalDate.now(), valida edad 14–120 y mantiene la relación con sus tamizajes.
`src/main/java/cent35/models/ResultadosTamizaje.java` M, R, D, V	Entidad de resultado uno a uno; almacena puntajes, categorías, semáforos y derivaciones, y ofrece ayudantes de observaciones/derivación. Su ayudante de semáforo no es el llamado por el servicio principal, que usa Tamizaje.
`src/main/java/cent35/models/Tamizaje.java` M, R, D, V	Entidad central de atención con código único, consentimiento, fecha y relaciones a personas/controles/resultado. Mantiene vínculos bidireccionales y reglas de IMC, presión, riesgo cardiovascular simplificado y semáforo global.
`src/main/java/cent35/models/TomaPresion.java` M, R, D, V	Entidad de presión que extiende Control; guarda número de toma, sistólica y diastólica, valida rangos 70–260/40–150 y sistólica>diastólica, y se vincula muchos a uno con Tamizaje.

Valores enumerados

Archivo y ramas	Responsabilidad
`build/classes/java/main/cent35/models/enums/AntecedentesFamiliares.class` M, R, D, V	Bytecode Java heredado de AntecedentesFamiliares.class, generado al compilar su fuente del mismo nombre. No se edita ni se toma como referencia del código actual; puede estar desactualizado respecto de src/.
`build/classes/java/main/cent35/models/enums/RolEnfermero.class` M, R, D, V	Bytecode Java heredado de RolEnfermero.class, generado al compilar su fuente del mismo nombre. No se edita ni se toma como referencia del código actual; puede estar desactualizado respecto de src/.
`build/classes/java/main/cent35/models/enums/SemaforoGlobal.class` M, R, D, V	Bytecode Java heredado de SemaforoGlobal.class, generado al compilar su fuente del mismo nombre. No se edita ni se toma como referencia del código actual; puede estar desactualizado respecto de src/.
`build/classes/java/main/cent35/models/enums/Sexo.class` M, R, D, V	Bytecode Java heredado de Sexo.class, generado al compilar su fuente del mismo nombre. No se edita ni se toma como referencia del código actual; puede estar desactualizado respecto de src/.
`build/classes/java/main/cent35/models/enums/Tabaquismo.class` M, R, D, V	Bytecode Java heredado de Tabaquismo.class, generado al compilar su fuente del mismo nombre. No se edita ni se toma como referencia del código actual; puede estar desactualizado respecto de src/.

Archivo y ramas	Responsabilidad
`build/classes/java/main/cent35/models/enums/TipoAntecedente.class` M, R, D, V	Bytecode Java heredado de TipoAntecedente.class, generado al compilar su fuente del mismo nombre. No se edita ni se toma como referencia del código actual; puede estar desactualizado respecto de src/.
`build/classes/java/main/cent35/models/enums/Turnos.class` M, R, D, V	Bytecode Java heredado de Turnos.class, generado al compilar su fuente del mismo nombre. No se edita ni se toma como referencia del código actual; puede estar desactualizado respecto de src/.
`src/main/java/cent35/models/enums/AntecedentesFamiliares.java` M, R, D, V	Enum NO/SEGUNDO_GRADO/PRIMER_GRADO, con descripciones de parentesco. Se usa en cuestionario y puntaje FINDRISC.
`src/main/java/cent35/models/enums/RolEnfermero.java` M, R, D, V	Enum ENFERMERO/SUPERVISOR para distinguir operador y supervisor en validaciones del alta completa.
`src/main/java/cent35/models/enums/SemaforoGlobal.java` M, R, D, V	Enum VERDE/AMARILLO/ROJO para persistir y publicar la clasificación global de una atención.
`src/main/java/cent35/models/enums/Sexo.java` M, R, D, V	Enum M/F/O para el sexo registrado del paciente. Se persiste como texto y se acepta en peticiones JSON/parámetros.
`src/main/java/cent35/models/enums/Tabaquismo.java` M, R, D, V	Enum NUNCA/EX/ACTUAL; ACTUAL suma un factor al riesgo cardiovascular simplificado.
`src/main/java/cent35/models/enums/TipoAntecedente.java` M, R, D, V	Enum DIABETES/HIPERTENSION/ENFERMEDAD_CARDIOVASCULAR/OTRO para tipos de antecedentes; el antecedente cardiovascular activa semáforo cardiovascular rojo.
`src/main/java/cent35/models/enums/Turnos.java` M, R, D, V	Enum MAÑANA/TARDE/VESPERTINO para el turno del personal. La Ñ forma parte del valor de API.

Repositorios JPA

Archivo y ramas	Responsabilidad
`build/classes/java/main/cent35/repositories/AntropometriaRepository.class` M, R, D, V	Bytecode Java heredado de AntropometriaRepository.class, generado al compilar su fuente del mismo nombre. No se edita ni se toma como referencia del código actual; puede estar desactualizado respecto de src/.
`build/classes/java/main/cent35/repositories/CuestionarioRepository.class` M, R, D, V	Bytecode Java heredado de CuestionarioRepository.class, generado al compilar su fuente del mismo nombre. No se edita ni se toma como referencia del código actual; puede estar desactualizado respecto de src/.
`build/classes/java/main/cent35/repositories/EnfermeroRepository.class` M, R, D, V	Bytecode Java heredado de EnfermeroRepository.class, generado al compilar su fuente del mismo nombre. No se edita ni se toma como referencia del código actual; puede estar desactualizado respecto de src/.

Archivo y ramas	Responsabilidad
`build/classes/java/main/cent35/repositories/PacienteRepository.class` M, R, D, V	Bytecode Java heredado de PacienteRepository.class, generado al compilar su fuente del mismo nombre. No se edita ni se toma como referencia del código actual; puede estar desactualizado respecto de src/.
`build/classes/java/main/cent35/repositories/ResultadosTamizajeRepository.class` M, R, D, V	Bytecode Java heredado de ResultadosTamizajeRepository.class, generado al compilar su fuente del mismo nombre. No se edita ni se toma como referencia del código actual; puede estar desactualizado respecto de src/.
`build/classes/java/main/cent35/repositories/TamizajeRepository.class` M, R, D, V	Bytecode Java heredado de TamizajeRepository.class, generado al compilar su fuente del mismo nombre. No se edita ni se toma como referencia del código actual; puede estar desactualizado respecto de src/.
`build/classes/java/main/cent35/repositories/TomaPresionRepository.class` M, R, D, V	Bytecode Java heredado de TomaPresionRepository.class, generado al compilar su fuente del mismo nombre. No se edita ni se toma como referencia del código actual; puede estar desactualizado respecto de src/.
`src/main/java/cent35/repositories/AntropometriaRepository.java` M, R, D, V	JpaRepository<Antropometria, Long> con CRUD persistente; marcado RepositoryRestResource para exportación automática de Spring Data REST.
`src/main/java/cent35/repositories/CuestionarioRepository.java` M, R, D, V	JpaRepository<Cuestionario, Long> usado para listas, búsqueda y alta de cuestionarios; también exportable mediante Spring Data REST.
`src/main/java/cent35/repositories/EnfermeroRepository.java` M, R, D, V	JpaRepository<Enfermero, Integer> usado para altas, listados y resolución de operador/supervisor; también exportable mediante Spring Data REST.
`src/main/java/cent35/repositories/PacienteRepository.java` M, R, D, V	JpaRepository<Paciente, Integer> usado para altas, listas y reutilización de pacientes; también exportable mediante Spring Data REST.
`src/main/java/cent35/repositories/ResultadosTamizajeRepository.java` M, R, D, V	JpaRepository<ResultadosTamizaje, Long>; agrega findByTamizajeId para recuperar o reutilizar el resultado único de una ficha. Exportable por Data REST.
`src/main/java/cent35/repositories/TamizajeRepository.java` M, R, D, V	JpaRepository<Tamizaje, Long>; agrega consulta JPQL existsByIdRegistro sobre el campo id_registro para prevenir duplicados, además de saveAndFlush del alta atómica. Exportable por Data REST.
`src/main/java/cent35/repositories/TomaPresionRepository.java` M, R, D, V	JpaRepository<TomaPresion, Long>; agrega listado por ficha ordenado por numeroToma y conteo para numerar nuevas tomas. Exportable por Data REST.

Servicios de aplicación y cálculo

Archivo y ramas	Responsabilidad
`build/classes/java/main/cent35/services/FindriscService.class` M, R, D, V	Bytecode Java heredado de FindriscService.class, generado al compilar su fuente del mismo nombre. No se edita ni se toma como referencia del código actual; puede estar desactualizado respecto de src/.
`src/main/java/cent35/services/FindriscService.java` M, R, D, V	Calcula el puntaje FINDRISC disponible por edad, IMC, actividad, frutas/verduras, medicación y parentesco; omite cintura/glucemia previa y clasifica categorías, con máximo actual 17.
`src/main/java/cent35/services/ResultadoTamizajeService.java` V	Centraliza el cálculo compartido del alta y recálculo: combina FindriscService con reglas de Tamizaje, actualiza semáforos/derivaciones/observaciones y vincula el resultado.
`src/main/java/cent35/services/TamizajeCompletoService.java` V	Coordina el alta completa en una transacción: valida y resuelve personas, construye controles, calcula antes de escribir y persiste el grafo con rollback si falla; devuelve TamizajeCompletoDTO.

Perfiles y configuración de recursos

Archivo y ramas	Responsabilidad
`src/main/resources/application-demo.properties` V	Perfil local demo: escucha 127.0.0.1, H2 en archivo .tamizaje-demo/tamizaje con compatibilidad MySQL, dialecto H2, actualización de esquema y consola H2 desactivada.
`src/main/resources/application.properties` M, R, D, V	Configuración base: puerto 8080, nombre 2026, conexión MySQL local 3307/tamizaje con root, driver/dialecto MySQL8 y esquema Hibernate ddl-auto=update.

Frontend Vue y bibliotecas locales

Archivo y ramas	Responsabilidad
`src/main/resources/static/css/app.css` V	Estilos propios sobre Bootstrap: identidad visual, menú, tarjetas, formularios y validaciones, navegación del asistente, revisión e informes; adapta la vista a móviles y oculta controles al imprimir, conservando datos y advertencias del resultado.
`src/main/resources/static/index.html` V	Página de entrada y plantilla Vue en español: estructura Bootstrap, navegación, asistente de siete pasos, revisión, informe, listados, fichas con pestañas, modales y atributos de accesibilidad. Importa recursos locales y monta la raíz #app.

Archivo y ramas	Responsabilidad
`src/main/resources/static/js/api.js` V	Cliente HTTP del mismo origen con prefijo /api: JSON, parámetros, respuestas vacías, errores de servidor, red y cancelación; trata 404 opcionales. Provee formatos, edades y etiquetas, validadores de persona/controles/cuestionario, validación por paso y construcción del DTO de alta completa sin modificar el borrador.
`src/main/resources/static/js/app.js` V	Aplicación Vue con Composition API: estado reactivo, carga y búsqueda de listados, navegación por fragmentos, borrador y avance de los siete pasos, guardado atómico mediante un POST, informes, modales de personas, consulta de fichas, cambios de presión y recálculo/impresión.
`src/main/resources/static/vendor/bootstrap/LICENSE` V	Licencia MIT de Bootstrap que acompaña los archivos CSS y JavaScript incluidos en el repositorio.
`src/main/resources/static/vendor/bootstrap/bootstrap.bundle.min.js` V	Distribución JavaScript local de Bootstrap 5.3.8 con dependencias incluidas. Expone bootstrap.Modal, sus eventos, gestión de apertura/cierre y el comportamiento interactivo de los componentes Bootstrap.
`src/main/resources/static/vendor/bootstrap/bootstrap.min.css` V	Hoja oficial minificada Bootstrap 5.3.8: grilla, formularios, botones, tablas, alertas, tarjetas, navegación y utilidades utilizadas por la plantilla. app.css se carga después para personalizarla.
`src/main/resources/static/vendor/versions.json` V	Manifiesto de procedencia y versiones fijadas de Vue y Bootstrap: URLs de paquetes, integridad npm de Vue y hashes SHA-256 de los archivos y licencias distribuidos. Permite verificar exactamente qué bibliotecas locales se incluyeron.
`src/main/resources/static/vendor/vue/LICENSE` V	Licencia MIT de Vue distribuida junto a su biblioteca. Conserva las condiciones y atribuciones aplicables al código de terceros.
`src/main/resources/static/vendor/vue/vue.esm-browser.prod.js` V	Distribución local de producción Vue 3.5.44 en módulo ES para navegador, con compilador de plantillas. Proporciona createApp, ref, reactive, computed, watch y demás funciones importadas por app.js; evita depender de CDN o de compilación npm.

Pruebas Java y JavaScript

Archivo y ramas	Responsabilidad
`src/test/java/cent35/TamizajeCompletoIntegrationTests.java` V	Integración MockMvc y base H2 en memoria del POST /api/tamizajes/completo: 28 ejecuciones contando parámetros. Verifica grafo completo y cintura, resultado equivalente al recálculo, reutilización/supervisor, duplicados y carrera con restricción única, validaciones, consentimiento, roles/personas ausentes y rollback por fallo del cálculo.

Archivo y ramas	Responsabilidad
`src/test/java/cent35/WebUiIntegrationTests.java` V	Integración Spring Boot/MockMvc con perfil demo y H2 en memoria: 9 ejecuciones contando casos parametrizados de raíz index.html, Vue y Bootstrap locales, tipos de contenido, lectura API del mismo origen y 404 de cuestionario para ficha inexistente.
`src/test/java/com/example/tamizaje/TamizajeApplicationTests.java` M, R, D, V	Prueba original de arranque del contexto Spring con la clase cent35.TamizajeApplication. No selecciona perfil demo ni reemplaza datasource, por lo que usa la configuración por defecto y puede necesitar MySQL.
`src/test/js/api.test.mjs` V	13 pruebas con node:test del cliente HTTP y utilidades: codificación de parámetros, conservación de false, DELETE vacío, errores HTTP/red/cancelación, 404 opcional, cuestionario y antecedentes, presión, fechas/edad, paciente, antropometría y formatos/colores.
`src/test/js/direct-form.test.mjs` V	10 pruebas del constructor del alta completa: DTO exacto, personas nuevas o existentes y combinaciones, ID válidos, consentimiento/código/nombres/turno, edad/sexo, medidas/presión/respuestas y ausencia de mutaciones o referencias compartidas con el borrador.
`src/test/js/wizard-form.test.mjs` V	11 pruebas de validateDirectRegistrationStep: admite borradores parciales, valida sólo el paso visible, contempla personas existentes, exige cintura en asistente y antecedentes condicionales, revalida toda la revisión y rechaza índices inválidos sin mutar el borrador.

Frontend React

Archivo y ramas	Responsabilidad
`frontend/.gitignore` R, D	Excluye node_modules/, dist/ y .env.local del control de versiones del frontend React. El código fuente y package-lock.json sí se conservan. Presente en ramas React y demo.
`frontend/README.md` R, D	Guía de la variante React con Bootstrap: pantallas, Node.js requerido, instalación npm ci, desarrollo en puerto 5173, proxy hacia Spring Boot, pruebas y build, más restricciones del backend. Presente en React y demo.
`frontend/index.html` R, D	Documento HTML inicial de Vite, en español, con metadatos y contenedor #root. Carga /src/main.jsx como módulo; no contiene la aplicación completa. Presente en React y demo.
`frontend/package-lock.json` R, D	Bloquea las versiones exactas y la integridad del árbol npm para que npm ci reproduzca la instalación. Resuelve React/React DOM 19.3.0, Bootstrap 5.3.8, React-Bootstrap 2.10.10, Vite 7.3.7 y Vitest 3.2.7. No es un archivo para escribir lógica de negocio. Presente en React y demo.

Archivo y ramas	Responsabilidad
`frontend/package.json` R, D	Define el proyecto privado ECMAScript tamizaje-frontend 1.0.0, scripts dev/build/preview/test y rangos de versiones para React, React DOM, Bootstrap, React-Bootstrap, Lucide, Vite, Vitest, jsdom y Testing Library. Presente en React y demo.
`frontend/src/App.jsx` R, D	Aplicación React en un archivo: componentes Field, Form, Badge, StatCard y Empty; estado y navegación de App; resumen, búsqueda y tablas de pacientes, enfermeros y tamizajes; modales de alta; ficha Detail con antropometría, tomas de presión, cuestionario, resultado FINDRISC e impresión. Cada alta o control usa una petición separada: no implementa el asistente ni el guardado integral atómico de la rama Vue. Presente en React y demo.
`frontend/src/App.test.jsx` R, D	Cinco pruebas Vitest con Testing Library y fetch simulado: recuperar cuestionario al reabrir ficha, bloquear presión invertida sin POST, mostrar error de conexión con reintento, registrar paciente y cerrar modal, y Escape con restauración de foco. No utiliza una base real. Presente en React y demo.
`frontend/src/api.js` R, D	Cliente fetch de /api del mismo origen. Codifica parámetros o envía JSON, interpreta respuestas JSON/texto y conserva mensajes HTTP de error. Incluye label para etiquetas, age para edad y questionnaire para normalizar booleanos y antecedentes antes del envío. Presente en React y demo.
`frontend/src/api.test.js` R, D	Cinco pruebas Vitest del contrato del cliente: parámetros codificados al crear paciente, preservación del mensaje de error, conversión de booleanos y limpieza de antecedentes, mantenimiento de antecedentes elegidos y cálculo de edad con reloj simulado. Presente en React y demo.
`frontend/src/main.jsx` R, D	Punto de entrada React: importa React y createRoot, Bootstrap CSS y luego los estilos propios; monta App dentro de React.StrictMode en #root. Usa React-Bootstrap para componentes interactivos, sin cargar el bundle JavaScript de Bootstrap por separado. Presente en React y demo.
`frontend/src/styles.css` R, D	Personaliza variables de Bootstrap y apariencia sanitaria verde; sidebar, cabecera, tarjetas, formularios, listados, fichas y modales. Incluye adaptación móvil y reglas @media print para imprimir el resultado sin navegación ni formularios. Presente en React y demo.
`frontend/vite.config.js` R, D	Activa @vitejs/plugin-react, fija desarrollo en puerto 5173 con strictPort y proxy /api hacia API_TARGET o http://127.0.0.1:8080; configura Vitest con jsdom. El proxy pertenece al servidor de desarrollo, no al build de producción. Presente en React y demo.

Distribución ZIP de la variante React

Archivo y ramas	Responsabilidad
`distribucion/README.md` D	Guía y vínculo de descarga del ZIP de demostración React/Bootstrap: Java 17, extracción completa, iniciadores Windows/macOS/Linux, puerto 8090 y base H2 persistida en datos/. Aclara que esta rama solo añade distribución sobre React. Exclusivo de la rama demo.
`distribucion/SHA256SUMS.txt` D	Checksum SHA-256 del archivo Tamizaje-bootstrap.zip: 4d3796093878f61d8973045896e35d844cdada96fe2918367ac1c8f430592f9f. Permite comprobar que la descarga coincide byte por byte con la versión documentada. Exclusivo de demo.
`distribucion/Tamizaje-bootstrap.zip` D	Paquete binario de prueba de 39.140.363 bytes. Contiene cuatro archivos bajo Tamizaje/: iniciar.sh, iniciar.cmd, LEEME.txt y tamizaje-demo.jar; el JAR incluye frontend React compilado, API y H2 2.1.214. Guarda datos fuera del ZIP, en datos/ al ejecutarlo. Es una instantánea React y no contiene el asistente Vue actual. Exclusivo de demo.

Artefactos generados heredados de build/

Archivo y ramas	Responsabilidad
`build/bootRunMainClassName` M, R, D, V	Salida de la tarea Gradle que registró la clase principal de bootRun en una compilación anterior; se regenera y no define por sí sola la configuración fuente.
`build/classes/java/main/cent35/TamizajeApplication.class` M, R, D, V	Bytecode Java heredado de TamizajeApplication.class, generado al compilar su fuente del mismo nombre. No se edita ni se toma como referencia del código actual; puede estar desactualizado respecto de src/.
`build/classes/java/test/com/example/tamizaje/TamizajeApplicationTests.class` M, R, D, V	Bytecode Java heredado de TamizajeApplicationTests.class, generado al compilar su fuente del mismo nombre. No se edita ni se toma como referencia del código actual; puede estar desactualizado respecto de src/.
`build/reports/tests/test/classes/com.example.tamizaje.TamizajeApplicationTests.html` M, R, D, V	Página HTML generada de un paquete o clase de la prueba original. Describe una ejecución histórica y no certifica el estado actual de las pruebas.
`build/reports/tests/test/css/base-style.css` M, R, D, V	Estilos del informe HTML generado de pruebas Gradle. No son estilos del frontend React ni Vue.
`build/reports/tests/test/css/style.css` M, R, D, V	Estilos del informe HTML generado de pruebas Gradle. No son estilos del frontend React ni Vue.
`build/reports/tests/test/index.html` M, R, D, V	Índice HTML generado de un informe histórico de pruebas Gradle. Sirve para navegar los resultados de esa ejecución, no para iniciar la interfaz del proyecto.

Archivo y ramas	Responsabilidad
`build/reports/tests/test/js/report.js` M, R, D, V	JavaScript de navegación y presentación del informe HTML generado de pruebas Gradle. No contiene lógica del sistema de tamizaje.
`build/reports/tests/test/packages/com.example.tamizaje.html` M, R, D, V	Página HTML generada de un paquete o clase de la prueba original. Describe una ejecución histórica y no certifica el estado actual de las pruebas.
`build/resources/main/application.properties` M, R, D, V	Copia procesada de la configuración base incluida en una compilación anterior. La configuración editable es src/main/resources/application.properties; esta copia puede estar desactualizada.
`build/test-results/test/TEST-com.example.tamizaje.TamizajeApplicationTests.xml` M, R, D, V	Resultado JUnit XML de una ejecución histórica de la prueba de contexto original. Lo consumen herramientas de informes; debe regenerarse para conocer un resultado actual.
`build/test-results/test/binary/output.bin` M, R, D, V	Datos o índice binario interno de resultados de pruebas Gradle (salida de consola y eventos). Se regeneran al ejecutar test; no se editan manualmente.
`build/test-results/test/binary/output.bin.idx` M, R, D, V	Datos o índice binario interno de resultados de pruebas Gradle (salida de consola y eventos). Se regeneran al ejecutar test; no se editan manualmente.
`build/test-results/test/binary/results.bin` M, R, D, V	Datos o índice binario interno de resultados de pruebas Gradle (salida de consola y eventos). Se regeneran al ejecutar test; no se editan manualmente.
`build/tmp/compileJava/previous-compilation-data.bin` M, R, D, V	Metadatos binarios internos de compilación incremental de Gradle. Son caché regenerable de una compilación anterior, no código fuente ni datos de pacientes.
`build/tmp/compileTestJava/previous-compilation-data.bin` M, R, D, V	Metadatos binarios internos de compilación incremental de Gradle. Son caché regenerable de una compilación anterior, no código fuente ni datos de pacientes.

Documentación nueva

Archivo y ramas	Responsabilidad
`docs/Tamizaje-documentacion-integral.pdf` V (documentación nueva)	Versión distribuible del manual, con texto seleccionable, índice con vínculos, marcadores, tablas y numeración de páginas. No ejecuta la aplicación.
`docs/documentacion-integral.md` V (documentación nueva)	Fuente editable del manual: funcionamiento, ramas, arquitectura, operación, API, inventario por archivo y anexos. Es la entrada de generar_pdf.py.
`docs/documentacion.css` V (documentación nueva)	Presentación del manual: portada, colores, tablas, bloques de código, saltos de página, índice y márgenes/pies A4. No afecta el CSS de la aplicación.

Archivo y ramas	Responsabilidad
`docs/generar_pdf.py` V (documentación nueva)	Generador reproducible: convierte el Markdown mediante Pandoc, aplica documentacion.css y usa Chromium/Chrome para exportar el PDF, sin arrancar Spring Boot ni modificar datos.
`docs/inventario-archivos.json` V (documentación nueva)	Inventario legible por herramientas: ramas y commits auditados, ruta y responsabilidad de cada archivo, pertenencia por rama y tipo de archivo. Complementa el catálogo del PDF.

8. Anexos y referencias

Contenido del ZIP de prueba

Las siguientes rutas son entradas dentro de `distribucion/Tamizaje-bootstrap.zip` o recursos web dentro de su JAR. No son archivos Git adicionales de la rama Vue. El JAR contiene además dependencias Java de terceros que se ejecutan junto al backend. Los nombres con `!` indican una ruta interna del JAR.

Entrada del paquete	Responsabilidad
`Tamizaje/iniciar.sh`	Iniciador Bash para macOS/Linux. Se ubica en su propia carpeta, verifica java y ejecuta el JAR en 127.0.0.1:8090 con conexión H2 de archivo ./datos/tamizaje, modo MySQL, usuario sa y dialecto H2. Los parámetros de arranque sustituyen la configuración MySQL incluida en el JAR.
`Tamizaje/iniciar.cmd`	Iniciador Windows equivalente: cambia a la carpeta del script, verifica java mediante where, lanza el mismo JAR/H2 en 127.0.0.1:8090 y conserva la ventana con pause para leer posibles errores.
`Tamizaje/LEEME.txt`	Instrucciones en texto: Java 17, extracción y ejecución, prueba guiada con datos ficticios, conservación de la carpeta datos/, cierre con Ctrl+C y manejo del puerto 8090 ocupado.
`Tamizaje/tamizaje-demo.jar`	JAR ejecutable Spring Boot de 43.451.520 bytes con clases Java, dependencias de servidor y base H2 y frontend React ya compilado. No necesita Node.js ni MySQL para la demo. Contiene los mismos conceptos de entidades/controladores del backend de la rama React, y no el alta transaccional de la rama Vue.
`Tamizaje/tamizaje-demo.jar!BOOT-INF/classes/static/index.html`	HTML producido por el build de Vite e integrado en el JAR para que Spring sirva la aplicación React en el mismo puerto de la API.
`Tamizaje/tamizaje-demo.jar!BOOT-INF/classes/static/assets/index-Dpb0-Wo5.css`	CSS generado/minificado que reúne Bootstrap y los estilos de la aplicación React para la demo.
`Tamizaje/tamizaje-demo.jar!BOOT-INF/classes/static/assets/index-B_BokLW2.js`	JavaScript generado/minificado de la aplicación React y sus dependencias de navegador; se recompila a partir de frontend/ y no se edita como fuente.

Glosario

Término	Significado en este proyecto
Tamizaje / ficha / atención	Registro central que vincula personas, consentimiento, controles y resultado.
DTO	Objeto de transferencia que define qué datos JSON entran o salen, evitando serializar todo el grafo JPA.

Término	Significado en este proyecto
JPA / Hibernate	Persistencia de entidades Java y sus relaciones en la base de datos.
Repository	Interfaz Spring Data para consultar y guardar entidades.
Transacción	Unidad de trabajo que confirma todas sus escrituras o las revierte ante un fallo.
Perfil demo	Configuración de Spring para probar con H2 local persistente. No es la misma rama que el ZIP React.
JAR ejecutable	Paquete Java con aplicación y dependencias, iniciado con `java -jar`.
Wrapper Gradle	Scripts y JAR que seleccionan y descargan la versión de Gradle fijada por el proyecto.
SPA	Interfaz que mantiene estado y cambia de vista en el navegador sin cargar una página HTML distinta por cada pantalla.
Mismo origen	Frontend y API comparten protocolo, host y puerto; en Vue los sirve Spring Boot.
Vendor	Bibliotecas externas incluidas con versión y licencias, distintas del código propio.
Código fuente / artefacto	Archivo editable que define el comportamiento / resultado producido por una compilación o empaquetado.

Fuentes y límites de la revisión

Las fuentes primarias fueron las cuatro referencias Git del capítulo 2, `git ls-tree`, el historial de commits, las clases Java, los archivos Vue/React, los manifiestos de dependencias, las pruebas y el contenido del ZIP. Se contrastaron las ramas remotas al momento de la revisión. Los comandos del manual son instrucciones de uso; la generación del PDF no vuelve a ejecutar por sí sola las pruebas de la aplicación.

La documentación conserva el código de referencia del 14 de octubre y se edita el 15 de octubre de 2026. Incluye comportamiento y límites observados en esas fuentes, sin prometer que futuros commits o cambios externos de base conserven el mismo resultado. La actualización del manual debe acompañar cualquier modificación del contrato, de reglas o de distribución.

Recursos de referencia: repositorio, rama Vue, rama React, rama de distribución, Spring Boot 2.7.14, Gradle 7.6.4, Vue y Bootstrap 5.3.